Model4

March 24, 2026

[1]:

```
---------------------------------------------------------------------------
ModuleNotFoundError                       Traceback (most recent call last)
Cell In[1], line 11
      8 from dataclasses import dataclass
      9 from typing import Dict, List, Tuple, Optional
---> 11 import numpy as np
     12 import torch
     13 import torch.nn as nn

ModuleNotFoundError: No module named 'numpy'
```

[2]:

```
!pip install numpy torch datasets transformers sentence-transformers
```

```
Requirement already satisfied: numpy in /opt/conda/lib/python3.13/site-packages
(2.4.3)
Requirement already satisfied: torch in /opt/conda/lib/python3.13/site-packages
(2.10.0)
Requirement already satisfied: datasets in /opt/conda/lib/python3.13/site-
packages (4.8.2)
Requirement already satisfied: transformers in /opt/conda/lib/python3.13/site-
packages (5.3.0)
Requirement already satisfied: sentence-transformers in
/opt/conda/lib/python3.13/site-packages (5.3.0)
Requirement already satisfied: filelock in /opt/conda/lib/python3.13/site-
packages (from torch) (3.25.2)
Requirement already satisfied: typing-extensions>=4.10.0 in
/opt/conda/lib/python3.13/site-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools in /opt/conda/lib/python3.13/site-
packages (from torch) (80.9.0)
Requirement already satisfied: sympy>=1.13.3 in /opt/conda/lib/python3.13/site-
packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in
/opt/conda/lib/python3.13/site-packages (from torch) (3.6.1)
Requirement already satisfied: jinja2 in /opt/conda/lib/python3.13/site-packages
(from torch) (3.1.6)
```

```
Requirement already satisfied: fsspec>=0.8.5 in /opt/conda/lib/python3.13/site-
packages (from torch) (2026.2.0)
Requirement already satisfied: cuda-bindings==12.9.4 in
/opt/conda/lib/python3.13/site-packages (from torch) (12.9.4)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93 in
/opt/conda/lib/python3.13/site-packages (from torch) (12.8.93)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90 in
/opt/conda/lib/python3.13/site-packages (from torch) (12.8.90)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90 in
/opt/conda/lib/python3.13/site-packages (from torch) (12.8.90)
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in
/opt/conda/lib/python3.13/site-packages (from torch) (9.10.2.21)
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1 in
/opt/conda/lib/python3.13/site-packages (from torch) (12.8.4.1)
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83 in
/opt/conda/lib/python3.13/site-packages (from torch) (11.3.3.83)
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90 in
/opt/conda/lib/python3.13/site-packages (from torch) (10.3.9.90)
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90 in
/opt/conda/lib/python3.13/site-packages (from torch) (11.7.3.90)
Requirement already satisfied: nvidia-cusparse-cu12==12.5.8.93 in
/opt/conda/lib/python3.13/site-packages (from torch) (12.5.8.93)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in
/opt/conda/lib/python3.13/site-packages (from torch) (0.7.1)
Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in
/opt/conda/lib/python3.13/site-packages (from torch) (2.27.5)
Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in
/opt/conda/lib/python3.13/site-packages (from torch) (3.4.5)
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90 in
/opt/conda/lib/python3.13/site-packages (from torch) (12.8.90)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93 in
/opt/conda/lib/python3.13/site-packages (from torch) (12.8.93)
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3 in
/opt/conda/lib/python3.13/site-packages (from torch) (1.13.1.3)
Requirement already satisfied: triton==3.6.0 in /opt/conda/lib/python3.13/site-
packages (from torch) (3.6.0)
Requirement already satisfied: cuda-pathfinder~=1.1 in
/opt/conda/lib/python3.13/site-packages (from cuda-bindings==12.9.4->torch)
(1.4.3)
Requirement already satisfied: pyarrow>=21.0.0 in
/opt/conda/lib/python3.13/site-packages (from datasets) (23.0.1)
Requirement already satisfied: dill<0.4.2,>=0.3.0 in
/opt/conda/lib/python3.13/site-packages (from datasets) (0.4.1)
Requirement already satisfied: pandas in /opt/conda/lib/python3.13/site-packages
(from datasets) (3.0.1)
Requirement already satisfied: requests>=2.32.2 in
/opt/conda/lib/python3.13/site-packages (from datasets) (2.32.5)
Requirement already satisfied: httpx<1.0.0 in /opt/conda/lib/python3.13/site-
```

```
packages (from datasets) (0.28.1)
Requirement already satisfied: tqdm>=4.66.3 in /opt/conda/lib/python3.13/site-
packages (from datasets) (4.67.1)
Requirement already satisfied: xxhash in /opt/conda/lib/python3.13/site-packages
(from datasets) (3.6.0)
Requirement already satisfied: multiprocess<0.70.20 in
/opt/conda/lib/python3.13/site-packages (from datasets) (0.70.19)
Requirement already satisfied: huggingface-hub<2.0,>=0.25.0 in
/opt/conda/lib/python3.13/site-packages (from datasets) (1.7.1)
Requirement already satisfied: packaging in /opt/conda/lib/python3.13/site-
packages (from datasets) (25.0)
Requirement already satisfied: pyyaml>=5.1 in /opt/conda/lib/python3.13/site-
packages (from datasets) (6.0.3)
Requirement already satisfied: aiohttp!=4.0.0a0,!=4.0.0a1 in
/opt/conda/lib/python3.13/site-packages (from
fsspec[http]<=2026.2.0,>=2023.1.0->datasets) (3.13.3)
Requirement already satisfied: anyio in /opt/conda/lib/python3.13/site-packages
(from httpx<1.0.0->datasets) (4.12.0)
Requirement already satisfied: certifi in /opt/conda/lib/python3.13/site-
packages (from httpx<1.0.0->datasets) (2026.1.4)
Requirement already satisfied: httpcore==1.* in /opt/conda/lib/python3.13/site-
packages (from httpx<1.0.0->datasets) (1.0.9)
Requirement already satisfied: idna in /opt/conda/lib/python3.13/site-packages
(from httpx<1.0.0->datasets) (3.11)
Requirement already satisfied: h11>=0.16 in /opt/conda/lib/python3.13/site-
packages (from httpcore==1.*->httpx<1.0.0->datasets) (0.16.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.4.2 in
/opt/conda/lib/python3.13/site-packages (from huggingface-
hub<2.0,>=0.25.0->datasets) (1.4.2)
Requirement already satisfied: typer in /opt/conda/lib/python3.13/site-packages
(from huggingface-hub<2.0,>=0.25.0->datasets) (0.24.1)
Requirement already satisfied: regex!=2019.12.17 in
/opt/conda/lib/python3.13/site-packages (from transformers) (2026.2.28)
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in
/opt/conda/lib/python3.13/site-packages (from transformers) (0.22.2)
Requirement already satisfied: safetensors>=0.4.3 in
/opt/conda/lib/python3.13/site-packages (from transformers) (0.7.0)
Requirement already satisfied: scikit-learn in /opt/conda/lib/python3.13/site-
packages (from sentence-transformers) (1.8.0)
Requirement already satisfied: scipy in /opt/conda/lib/python3.13/site-packages
(from sentence-transformers) (1.17.1)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in
/opt/conda/lib/python3.13/site-packages (from
aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2026.2.0,>=2023.1.0->datasets) (2.6.1)
Requirement already satisfied: aiosignal>=1.4.0 in
/opt/conda/lib/python3.13/site-packages (from
aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2026.2.0,>=2023.1.0->datasets) (1.4.0)
Requirement already satisfied: attrs>=17.3.0 in /opt/conda/lib/python3.13/site-
```

```
packages (from
aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2026.2.0,>=2023.1.0->datasets)
(25.4.0)
Requirement already satisfied: frozenlist>=1.1.1 in
/opt/conda/lib/python3.13/site-packages (from
aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2026.2.0,>=2023.1.0->datasets) (1.8.0)
Requirement already satisfied: multidict<7.0,>=4.5 in
/opt/conda/lib/python3.13/site-packages (from
aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2026.2.0,>=2023.1.0->datasets) (6.7.1)
Requirement already satisfied: propcache>=0.2.0 in
/opt/conda/lib/python3.13/site-packages (from
aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2026.2.0,>=2023.1.0->datasets) (0.4.1)
Requirement already satisfied: yarl<2.0,>=1.17.0 in
/opt/conda/lib/python3.13/site-packages (from
aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2026.2.0,>=2023.1.0->datasets)
(1.23.0)
Requirement already satisfied: charset_normalizer<4,>=2 in
/opt/conda/lib/python3.13/site-packages (from requests>=2.32.2->datasets)
(3.4.4)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/opt/conda/lib/python3.13/site-packages (from requests>=2.32.2->datasets)
(2.6.2)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in
/opt/conda/lib/python3.13/site-packages (from sympy>=1.13.3->torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in
/opt/conda/lib/python3.13/site-packages (from jinja2->torch) (3.0.3)
Requirement already satisfied: python-dateutil>=2.8.2 in
/opt/conda/lib/python3.13/site-packages (from pandas->datasets) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /opt/conda/lib/python3.13/site-
packages (from python-dateutil>=2.8.2->pandas->datasets) (1.17.0)
Requirement already satisfied: joblib>=1.3.0 in /opt/conda/lib/python3.13/site-
packages (from scikit-learn->sentence-transformers) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.2.0 in
/opt/conda/lib/python3.13/site-packages (from scikit-learn->sentence-
transformers) (3.6.0)
Requirement already satisfied: click>=8.2.1 in /opt/conda/lib/python3.13/site-
packages (from typer->huggingface-hub<2.0,>=0.25.0->datasets) (8.3.1)
Requirement already satisfied: shellingham>=1.3.0 in
/opt/conda/lib/python3.13/site-packages (from typer->huggingface-
hub<2.0,>=0.25.0->datasets) (1.5.4)
Requirement already satisfied: rich>=12.3.0 in /opt/conda/lib/python3.13/site-
packages (from typer->huggingface-hub<2.0,>=0.25.0->datasets) (14.3.3)
Requirement already satisfied: annotated-doc>=0.0.2 in
/opt/conda/lib/python3.13/site-packages (from typer->huggingface-
hub<2.0,>=0.25.0->datasets) (0.0.4)
Requirement already satisfied: markdown-it-py>=2.2.0 in
/opt/conda/lib/python3.13/site-packages (from rich>=12.3.0->typer->huggingface-
hub<2.0,>=0.25.0->datasets) (4.0.0)
```

```
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in
/opt/conda/lib/python3.13/site-packages (from rich>=12.3.0->typer->huggingface-
hub<2.0,>=0.25.0->datasets) (2.19.2)
Requirement already satisfied: mdurl~=0.1 in /opt/conda/lib/python3.13/site-
packages (from markdown-it-py>=2.2.0->rich>=12.3.0->typer->huggingface-
hub<2.0,>=0.25.0->datasets) (0.1.2)
```

[4]:
```python
import os
import re
import json
import math
import time
import random
import shutil
from dataclasses import dataclass
from typing import Dict, List, Tuple, Optional

import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim

from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForCausalLM
from sentence_transformers import SentenceTransformer


# ================================================================
# Config
# ================================================================

@dataclass
class Config:
    # Dataset
    train_subset: int = 300
    test_subset: int = 100

    # Models
    llm_name: str = "Qwen/Qwen2.5-1.5B-Instruct"
    # llm_name: str = "Qwen/Qwen2.5-3B-Instruct"
    embed_name: str = "sentence-transformers/all-MiniLM-L6-v2"

    # Oracle search: denser around strong region
    oracle_budgets: Tuple[int, ...] = (
        96, 104, 112, 120, 128, 136, 144, 152, 160, 176, 192, 208, 220
    )
```

```
    # Supervised warm-start
    warmstart_epochs: int = 15
    warmstart_learning_rate: float = 5e-4

    # RL fine-tuning
    rl_epochs: int = 5
    policy_learning_rate: float = 3e-4
    value_learning_rate: float = 8e-4
    entropy_coef: float = 0.002
    value_loss_coef: float = 0.5
    grad_clip_norm: float = 1.0

    # Reward
    reward_token_penalty: float = 0.00005
    use_soft_reward: bool = False
    soft_reward_floor: float = 0.0
    exact_match_bonus: float = 0.25

    # Thinking budget range
    min_budget: int = 64
    max_budget: int = 220

    # Final answer generation budget
    answer_max_new_tokens: int = 20

    # Generation
    do_sample: bool = False
    temperature: float = 0.0

    # RL exploration warmup
    warmup_epochs_rl: int = 2
    warmup_mix_rl: float = 0.25
    warmup_min_budget_high: int = 128

    # Hardware
    seed: int = 42
    device: str = "cuda" if torch.cuda.is_available() else "cpu"

    # Logging
    print_every: int = 20
    debug_examples: int = 3
    eval_print_every: int = 10

    # Cache / outputs
    cache_dir: str = "./cache_rl_two_stage_v9"
    clear_cache_on_start: bool = True
    oracle_cache_path: str = "oracle_budget_labels_v9.json"
```

```
    # Saved models
    best_warmstart_policy_path: str = "best_warmstart_policy_v9.pt"
    best_warmstart_policy_by_acc_path: str = "best_warmstart_policy_by_acc_v9.
 ↪pt"
    best_policy_path: str = "best_policy_v9.pt"
    best_policy_by_acc_path: str = "best_policy_by_acc_v9.pt"
    best_value_path: str = "best_value_v9.pt"
    final_policy_path: str = "two_stage_continuous_budget_policy_v9_final.pt"
    final_value_path: str = "two_stage_value_net_v9_final.pt"

    # Baselines
    baseline_budgets: Tuple[int, ...] = (96, 128, 144, 160, 176, 192, 220)


CFG = Config()


# ============================================================
# Utilities
# ============================================================

def set_seed(seed: int) -> None:
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)


def ensure_dir(path: str) -> None:
    os.makedirs(path, exist_ok=True)


def reset_cache_dir(cache_dir: str, retries: int = 5, delay: float = 0.5) ->␣
 ↪None:
    """
    More robust cache reset for environments where directory handles may linger.
    """
    if not os.path.exists(cache_dir):
        os.makedirs(cache_dir, exist_ok=True)
        return

    last_error = None

    for _ in range(retries):
        try:
```

```
            if os.path.exists(cache_dir):
                shutil.rmtree(cache_dir, ignore_errors=False)

            if os.path.exists(cache_dir):
                for root, dirs, files in os.walk(cache_dir, topdown=False):
                    for file_name in files:
                        file_path = os.path.join(root, file_name)
                        try:
                            os.remove(file_path)
                        except FileNotFoundError:
                            pass
                    for dir_name in dirs:
                        dir_path = os.path.join(root, dir_name)
                        try:
                            os.rmdir(dir_path)
                        except OSError:
                            pass
                if os.path.exists(cache_dir):
                    os.rmdir(cache_dir)

            break
        except OSError as e:
            last_error = e
            time.sleep(delay)

    if os.path.exists(cache_dir):
        raise RuntimeError(
            f"Failed to fully clear cache directory after retries: {cache_dir}"
        ) from last_error

    os.makedirs(cache_dir, exist_ok=True)


def clamp_budget(x: float, min_budget: int, max_budget: int) -> int:
    return int(max(min_budget, min(max_budget, round(x))))


def denormalize_budget(x: float, min_budget: int, max_budget: int) -> float:
    return x * (max_budget - min_budget) + min_budget


def budget_to_normalized(budget: int, min_budget: int, max_budget: int) ->␣
 ↪float:
    return float(budget - min_budget) / float(max_budget - min_budget)


def normalize_scalar_list(values: List[float]) -> List[float]:
```

```
    if len(values) == 0:
        return values
    arr = np.array(values, dtype=np.float32)
    mean = float(arr.mean())
    std = float(arr.std()) + 1e-8
    return ((arr - mean) / std).tolist()


def clean_model_output(text: str) -> str:
    if text is None:
        return ""

    cleaned = text

    stop_markers = [
        "\nHuman:",
        "\nAssistant:",
        "\nUser:",
        "\nSystem:",
        "Human:",
        "Assistant:",
        "User:",
        "System:",
        "You are an AI assistant",
        "Provide a detailed answer",
        "I'll respond in English",
        "<|im_end|>",
        "<|endoftext|>",
    ]

    for marker in stop_markers:
        if marker in cleaned:
            cleaned = cleaned.split(marker)[0]

    return cleaned.strip()


# ============================================================
# GSM8K helpers
# ============================================================

def extract_gsm8k_final_answer(answer_text: str) -> str:
    if "####" in answer_text:
        return answer_text.split("####")[-1].strip()
    return answer_text.strip()
```

```python
def normalize_numeric_string(text: str) -> str:
    if text is None:
        return ""

    text = text.strip()
    text = text.replace(",", "")
    text = text.replace("$", "")
    text = text.replace("%", "")
    text = text.strip()

    matches = re.findall(r"-?\d+(?:\.\d+)?", text)
    if not matches:
        return text.lower().strip()
    return matches[-1]


def parse_final_answer(output_text: str) -> str:
    text = clean_model_output(output_text)

    patterns = [
        r"FINAL ANSWER\s*:\s*(-?\d+(?:\.\d+)?)",
        r"Final Answer\s*:\s*(-?\d+(?:\.\d+)?)",
        r"Answer\s*:\s*(-?\d+(?:\.\d+)?)",
    ]

    for pattern in patterns:
        m = re.search(pattern, text, re.IGNORECASE)
        if m:
            return normalize_numeric_string(m.group(1))

    lines = [line.strip() for line in text.splitlines() if line.strip()]
    first_line = lines[0] if lines else text.strip()

    m = re.search(r"-?\d+(?:\.\d+)?", first_line)
    if m:
        return normalize_numeric_string(m.group(0))

    matches = re.findall(r"-?\d+(?:\.\d+)?", text)
    if matches:
        return normalize_numeric_string(matches[0])

    return normalize_numeric_string(text)


def is_correct_prediction(pred: str, gold: str) -> int:
    return int(pred == gold)
```

```python
def soft_numeric_match(pred: str, gold: str, floor: float = 0.0) -> float:
    try:
        p = float(pred)
        g = float(gold)
        error = abs(p - g)
        denom = max(abs(g), 1.0)
        score = 1.0 - (error / denom)
        return float(max(floor, min(1.0, score)))
    except Exception:
        return 0.0


def compute_reward(
    pred: str,
    gold: str,
    thinking_tokens_used: int,
    cfg: Config
) -> Tuple[float, float]:
    exact = float(is_correct_prediction(pred, gold))

    if cfg.use_soft_reward:
        answer_score = max(exact, soft_numeric_match(pred, gold, floor=cfg.
 ↪soft_reward_floor))
    else:
        answer_score = exact

    reward = answer_score - cfg.reward_token_penalty *␣
 ↪float(thinking_tokens_used)

    if exact > 0.5:
        reward += cfg.exact_match_bonus

    return reward, exact


# ============================================================
# Prompts
# ============================================================

def build_thinking_prompt(question: str) -> str:
    return (
        "Solve the following grade-school math problem carefully.\n"
        "Reason step by step.\n"
        "Keep the reasoning concise and relevant.\n"
        "Do not start a conversation.\n"
        "Do not add extra instructions.\n"
```

```
        "Only produce reasoning.\n\n"
        f"Question: {question}\n\n"
        "Reasoning:"
    )


def build_answer_prompt(question: str, thinking_text: str) -> str:
    return (
        "Use the reasoning below to produce the final numeric answer.\n"
        "Return only one line in this exact format:\n"
        "FINAL ANSWER: <number>\n\n"
        f"Question: {question}\n\n"
        f"Reasoning:\n{thinking_text}\n\n"
        "FINAL ANSWER:"
    )


# ============================================================
# Featurizer
# ============================================================

class QuestionFeaturizer:
    def __init__(self, embed_model_name: str, device: str):
        self.embedder = SentenceTransformer(embed_model_name, device=device)

    def handcrafted_features(self, question: str) -> np.ndarray:
        q = question.lower()
        tokens = q.split()

        length_feat = len(tokens)
        num_count = len(re.findall(r"\d+", q))
        sentence_len_chars = len(q)
        sentence_count = max(1, len(re.findall(r"[.!?]", q)))

        keywords = [
            "total", "left", "remaining", "each", "every",
            "more", "less", "after", "before", "twice",
            "half", "times", "altogether", "difference",
            "sum", "product", "cost", "price", "sold",
            "per", "average", "equal", "share", "then",
            "gave", "bought", "earned", "minutes", "hours",
            "days", "weeks", "fraction", "ratio", "percent",
            "another", "still", "now", "together"
        ]
        keyword_hits = [1.0 if kw in q else 0.0 for kw in keywords]
```

```
        plus_cue = 1.0 if any(w in q for w in ["more", "sum", "altogether",␣
↪"total", "together"]) else 0.0
        minus_cue = 1.0 if any(w in q for w in ["left", "remaining",␣
↪"difference", "less", "still"]) else 0.0
        mult_cue = 1.0 if any(w in q for w in ["each", "every", "times",␣
↪"product", "twice"]) else 0.0
        div_cue = 1.0 if any(w in q for w in ["per", "average", "equally",␣
↪"half", "share", "ratio"]) else 0.0
        multi_step_cue = 1.0 if any(w in q for w in ["then", "after", "before",␣
↪"altogether", "remaining", "now"]) else 0.0

        unit_time_cue = 1.0 if any(w in q for w in ["minutes", "hours", "days",␣
↪"weeks"]) else 0.0
        money_cue = 1.0 if "$" in q or any(w in q for w in ["dollar",␣
↪"dollars", "cost", "price"]) else 0.0
        fraction_cue = 1.0 if any(w in q for w in ["half", "fraction",␣
↪"percent", "ratio"]) else 0.0

        feats = np.array(
            [
                length_feat,
                num_count,
                sentence_len_chars,
                sentence_count,
                plus_cue,
                minus_cue,
                mult_cue,
                div_cue,
                multi_step_cue,
                unit_time_cue,
                money_cue,
                fraction_cue,
            ] + keyword_hits,
            dtype=np.float32
        )
        return feats

    def encode(self, question: str) -> np.ndarray:
        emb = self.embedder.encode(
            question,
            convert_to_numpy=True,
            normalize_embeddings=True
        ).astype(np.float32)
        hand = self.handcrafted_features(question)
        return np.concatenate([emb, hand], axis=0)
```

```python
# ============================================================
# Policy + Value Networks
# ============================================================

class ContinuousBudgetPolicy(nn.Module):
    def __init__(self, input_dim: int, hidden_dim: int = 256):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
        )
        self.mean_head = nn.Linear(hidden_dim, 1)
        self.log_std = nn.Parameter(torch.tensor([0.0], dtype=torch.float32))

    def forward(self, x: torch.Tensor) -> Tuple[torch.Tensor, torch.Tensor]:
        h = self.net(x)
        mean = self.mean_head(h)
        log_std = self.log_std.expand_as(mean)
        return mean, log_std

    def sample_budget(self, x: torch.Tensor) -> Tuple[torch.Tensor, torch.
 ↪Tensor, torch.Tensor]:
        mean, log_std = self.forward(x)
        std = torch.exp(log_std)
        dist = torch.distributions.Normal(mean, std)
        raw = dist.rsample()
        normalized = torch.sigmoid(raw)
        log_prob = dist.log_prob(raw).sum(dim=-1)
        entropy = dist.entropy().sum(dim=-1)
        return normalized, log_prob, entropy

    def deterministic_budget(self, x: torch.Tensor) -> torch.Tensor:
        mean, _ = self.forward(x)
        return torch.sigmoid(mean)


class ValueNetwork(nn.Module):
    def __init__(self, input_dim: int, hidden_dim: int = 256):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
```

```
            nn.Linear(hidden_dim, 1),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.net(x).squeeze(-1)


# ============================================================
# LLM wrapper
# ============================================================

class LLMReasoner:
    def __init__(self, model_name: str, device: str):
        self.tokenizer = AutoTokenizer.from_pretrained(model_name)

        if self.tokenizer.pad_token is None:
            self.tokenizer.pad_token = self.tokenizer.eos_token

        dtype = torch.float16 if device == "cuda" else torch.float32

        if device == "cuda":
            self.model = AutoModelForCausalLM.from_pretrained(
                model_name,
                dtype=dtype
            ).to(device)
        else:
            self.model = AutoModelForCausalLM.from_pretrained(
                model_name,
                dtype=dtype
            )

        self.model.eval()
        self.device = device

        if hasattr(self.model, "generation_config") and self.model.
 ↪generation_config is not None:
            self.model.generation_config.do_sample = False
            self.model.generation_config.temperature = None
            self.model.generation_config.top_p = None
            self.model.generation_config.top_k = None

    def generate(
        self,
        prompt: str,
        max_new_tokens: int,
        do_sample: bool = False,
        temperature: float = 0.0
```

```
    ) -> Dict:
        inputs = self.tokenizer(prompt, return_tensors="pt").to(self.device)
        input_len = inputs["input_ids"].shape[1]

        gen_kwargs = {
            "max_new_tokens": max_new_tokens,
            "do_sample": do_sample,
            "pad_token_id": self.tokenizer.eos_token_id,
            "eos_token_id": self.tokenizer.eos_token_id,
        }

        if do_sample:
            gen_kwargs["temperature"] = temperature

        with torch.no_grad():
            outputs = self.model.generate(**inputs, **gen_kwargs)

        gen_ids = outputs[0][input_len:]
        text = self.tokenizer.decode(gen_ids, skip_special_tokens=True)
        text = clean_model_output(text)
        tokens_used = len(gen_ids)

        return {"text": text, "tokens_used": tokens_used}


# ============================================================
# Caching
# ============================================================

def cache_key(stage: str, question: str, extra: str, model_name: str) -> str:
    import hashlib
    raw = f"{stage}|||{model_name}|||{question}|||{extra}"
    return hashlib.md5(raw.encode("utf-8")).hexdigest()


def load_from_cache(cache_dir: str, key: str) -> Optional[Dict]:
    path = os.path.join(cache_dir, f"{key}.json")
    if os.path.exists(path):
        with open(path, "r", encoding="utf-8") as f:
            return json.load(f)
    return None


def save_to_cache(cache_dir: str, key: str, value: Dict) -> None:
    path = os.path.join(cache_dir, f"{key}.json")
    with open(path, "w", encoding="utf-8") as f:
        json.dump(value, f, ensure_ascii=False, indent=2)
```

```python
# ============================================================
# Two-stage example run
# ============================================================

def run_two_stage_example(
    question: str,
    gold_answer: str,
    thinking_budget: int,
    llm: LLMReasoner,
    cfg: Config,
) -> Dict:
    ensure_dir(cfg.cache_dir)

    think_prompt = build_thinking_prompt(question)
    think_key = cache_key("thinking", question, f"budget={thinking_budget}",␣
 ↪cfg.llm_name)

    cached_think = load_from_cache(cfg.cache_dir, think_key)
    if cached_think is not None:
        thinking_text = clean_model_output(cached_think["text"])
        thinking_tokens_used = cached_think["tokens_used"]
    else:
        think_result = llm.generate(
            prompt=think_prompt,
            max_new_tokens=thinking_budget,
            do_sample=cfg.do_sample,
            temperature=cfg.temperature
        )
        thinking_text = clean_model_output(think_result["text"])
        thinking_tokens_used = think_result["tokens_used"]
        save_to_cache(cfg.cache_dir, think_key, {"text": thinking_text,␣
 ↪"tokens_used": thinking_tokens_used})

    answer_prompt = build_answer_prompt(question, thinking_text)
    answer_key = cache_key("answer", question, thinking_text, cfg.llm_name)

    cached_answer = load_from_cache(cfg.cache_dir, answer_key)
    if cached_answer is not None:
        answer_text = clean_model_output(cached_answer["text"])
        answer_tokens_used = cached_answer["tokens_used"]
    else:
        answer_result = llm.generate(
            prompt=answer_prompt,
            max_new_tokens=cfg.answer_max_new_tokens,
            do_sample=False,
```

```
            temperature=0.0
        )
        answer_text = clean_model_output(answer_result["text"])
        answer_tokens_used = answer_result["tokens_used"]
        save_to_cache(cfg.cache_dir, answer_key, {"text": answer_text,␣
 ↪"tokens_used": answer_tokens_used})

    pred = parse_final_answer(answer_text)
    reward, exact = compute_reward(pred, gold_answer, thinking_tokens_used, cfg)

    return {
        "prediction": pred,
        "gold": gold_answer,
        "correct": int(exact),
        "thinking_tokens_used": thinking_tokens_used,
        "answer_tokens_used": answer_tokens_used,
        "reward": reward,
        "thinking_text": thinking_text,
        "answer_text": answer_text,
        "thinking_budget": thinking_budget,
    }


# ============================================================
# Debug
# ============================================================

def print_debug_examples(
    data: List[Dict],
    llm: LLMReasoner,
    cfg: Config,
    budget: int,
    n: int = 3
) -> None:
    print("\n" + "=" * 100)
    print(f"DEBUG EXAMPLES WITH FIXED THINKING BUDGET = {budget}")
    print("=" * 100)

    for i, item in enumerate(data[:n]):
        question = item["question"]
        gold =␣
 ↪normalize_numeric_string(extract_gsm8k_final_answer(item["answer"]))
        result = run_two_stage_example(question, gold, budget, llm, cfg)

        print(f"\n[Example {i+1}]")
        print("-" * 100)
        print("QUESTION:")
```

```
        print(question)
        print("\nTHINKING TEXT:")
        print(result["thinking_text"])
        print("\nANSWER TEXT:")
        print(result["answer_text"])
        print("\nPARSED PREDICTION:", result["prediction"])
        print("GOLD ANSWER      :", gold)
        print("CORRECT          :", result["correct"])
        print("THINK TOKENS     :", result["thinking_tokens_used"])
        print("REWARD           :", result["reward"])
        print("-" * 100)


# ============================================================
# Baselines and evaluation
# ============================================================

def evaluate_fixed_budget(
    data: List[Dict],
    llm: LLMReasoner,
    cfg: Config,
    fixed_budget: int,
) -> Dict:
    corrects = []
    thinking_tokens = []
    rewards = []

    print(f"\nEvaluating fixed budget = {fixed_budget} on {len(data)} examples..
 ↪.")

    for idx, item in enumerate(data):
        question = item["question"]
        gold =␣
 ↪normalize_numeric_string(extract_gsm8k_final_answer(item["answer"]))
        result = run_two_stage_example(question, gold, fixed_budget, llm, cfg)

        corrects.append(result["correct"])
        thinking_tokens.append(result["thinking_tokens_used"])
        rewards.append(result["reward"])

        if (idx + 1) % cfg.eval_print_every == 0 or (idx + 1) == len(data):
            print(
                f"  Fixed budget {fixed_budget} | Done {idx+1}/{len(data)} | "
                f"Acc={np.mean(corrects):.4f} | AvgThinkTokens={np.
 ↪mean(thinking_tokens):.2f} | "
                f"AvgReward={np.mean(rewards):.4f}"
            )
```

```
    return {
        "accuracy": float(np.mean(corrects)) if corrects else 0.0,
        "avg_thinking_tokens": float(np.mean(thinking_tokens)) if␣
 ↪thinking_tokens else 0.0,
        "avg_reward": float(np.mean(rewards)) if rewards else 0.0,
        "budget": fixed_budget,
    }


def evaluate_policy_deterministic(
    data: List[Dict],
    featurizer: QuestionFeaturizer,
    policy: ContinuousBudgetPolicy,
    llm: LLMReasoner,
    cfg: Config,
) -> Dict:
    policy.eval()

    corrects = []
    thinking_tokens = []
    rewards = []
    budgets = []

    with torch.no_grad():
        for idx, item in enumerate(data):
            question = item["question"]
            gold =␣
 ↪normalize_numeric_string(extract_gsm8k_final_answer(item["answer"]))
            feat = featurizer.encode(question)
            x = torch.tensor(feat, dtype=torch.float32, device=cfg.device).
 ↪unsqueeze(0)

            norm_budget = policy.deterministic_budget(x).item()
            budget = clamp_budget(
                denormalize_budget(norm_budget, cfg.min_budget, cfg.max_budget),
                cfg.min_budget,
                cfg.max_budget,
            )

            result = run_two_stage_example(question, gold, budget, llm, cfg)
            corrects.append(result["correct"])
            thinking_tokens.append(result["thinking_tokens_used"])
            rewards.append(result["reward"])
            budgets.append(budget)

            if (idx + 1) % cfg.eval_print_every == 0 or (idx + 1) == len(data):
```

```
                print(
                    f"  Deterministic policy eval | Done {idx+1}/{len(data)} | "
                    f"Acc={np.mean(corrects):.4f} | AvgBudget={np.mean(budgets):
 ↪.2f}"
                )

    return {
        "accuracy": float(np.mean(corrects)) if corrects else 0.0,
        "avg_thinking_tokens": float(np.mean(thinking_tokens)) if␣
 ↪thinking_tokens else 0.0,
        "avg_reward": float(np.mean(rewards)) if rewards else 0.0,
        "avg_budget": float(np.mean(budgets)) if budgets else 0.0,
        "min_budget": float(np.min(budgets)) if budgets else 0.0,
        "max_budget": float(np.max(budgets)) if budgets else 0.0,
        "std_budget": float(np.std(budgets)) if budgets else 0.0,
    }


def evaluate_policy_sampled(
    data: List[Dict],
    featurizer: QuestionFeaturizer,
    policy: ContinuousBudgetPolicy,
    llm: LLMReasoner,
    cfg: Config,
) -> Dict:
    policy.eval()

    corrects = []
    thinking_tokens = []
    rewards = []
    budgets = []

    with torch.no_grad():
        for idx, item in enumerate(data):
            question = item["question"]
            gold =␣
 ↪normalize_numeric_string(extract_gsm8k_final_answer(item["answer"]))
            feat = featurizer.encode(question)
            x = torch.tensor(feat, dtype=torch.float32, device=cfg.device).
 ↪unsqueeze(0)

            norm_budget, _, _ = policy.sample_budget(x)
            budget = clamp_budget(
                denormalize_budget(norm_budget.item(), cfg.min_budget, cfg.
 ↪max_budget),
                cfg.min_budget,
                cfg.max_budget,
```

```
            )

            result = run_two_stage_example(question, gold, budget, llm, cfg)
            corrects.append(result["correct"])
            thinking_tokens.append(result["thinking_tokens_used"])
            rewards.append(result["reward"])
            budgets.append(budget)

            if (idx + 1) % cfg.eval_print_every == 0 or (idx + 1) == len(data):
                print(
                    f"  Sampled policy eval | Done {idx+1}/{len(data)} | "
                    f"Acc={np.mean(corrects):.4f} | AvgBudget={np.mean(budgets):
 ↪.2f}"
                )

    return {
        "accuracy": float(np.mean(corrects)) if corrects else 0.0,
        "avg_thinking_tokens": float(np.mean(thinking_tokens)) if␣
 ↪thinking_tokens else 0.0,
        "avg_reward": float(np.mean(rewards)) if rewards else 0.0,
        "avg_budget": float(np.mean(budgets)) if budgets else 0.0,
        "min_budget": float(np.min(budgets)) if budgets else 0.0,
        "max_budget": float(np.max(budgets)) if budgets else 0.0,
        "std_budget": float(np.std(budgets)) if budgets else 0.0,
    }


# ============================================================
# Oracle budget construction
# ============================================================

def build_oracle_budget_labels(
    train_data: List[Dict],
    llm: LLMReasoner,
    cfg: Config,
) -> List[Dict]:
    if os.path.exists(cfg.oracle_cache_path):
        print(f"\nLoading oracle labels from {cfg.oracle_cache_path} ...")
        with open(cfg.oracle_cache_path, "r", encoding="utf-8") as f:
            return json.load(f)

    print("\nBuilding oracle budget labels...")

    oracle_records = []

    for idx, item in enumerate(train_data):
        question = item["question"]
```

```
        gold = normalize_numeric_string(extract_gsm8k_final_answer(item["answer"]))

        best_budget = None
        best_reward = -1e9
        budget_rewards = {}

        for budget in cfg.oracle_budgets:
            result = run_two_stage_example(question, gold, budget, llm, cfg)
            reward = float(result["reward"])
            budget_rewards[str(budget)] = {
                "reward": reward,
                "correct": int(result["correct"]),
                "thinking_tokens_used": int(result["thinking_tokens_used"]),
                "prediction": result["prediction"],
            }

            if reward > best_reward:
                best_reward = reward
                best_budget = budget

        oracle_records.append(
            {
                "question": question,
                "gold_answer": gold,
                "best_budget": int(best_budget),
                "best_reward": float(best_reward),
                "budget_rewards": budget_rewards,
            }
        )

        if (idx + 1) % cfg.eval_print_every == 0 or (idx + 1) == len(train_data):
            print(
                f"  Oracle labels | Done {idx+1}/{len(train_data)} | "
                f"AvgBestBudget={np.mean([r['best_budget'] for r in oracle_records]):.2f} | "
                f"AvgBestReward={np.mean([r['best_reward'] for r in oracle_records]):.4f}"
            )

    with open(cfg.oracle_cache_path, "w", encoding="utf-8") as f:
        json.dump(oracle_records, f, ensure_ascii=False, indent=2)

    print(f"Saved oracle labels to {cfg.oracle_cache_path}")
    return oracle_records
```

```python
# ============================================================
# Supervised warm-start
# ============================================================

def warmstart_policy_from_oracle(
    oracle_records: List[Dict],
    test_data: List[Dict],
    featurizer: QuestionFeaturizer,
    policy: ContinuousBudgetPolicy,
    llm: LLMReasoner,
    cfg: Config,
) -> None:
    print("\nStarting supervised warm-start from oracle budgets...")

    optimizer = optim.Adam(policy.parameters(), lr=cfg.warmstart_learning_rate)
    mse_loss = nn.MSELoss()

    best_eval_reward = -1e9
    best_eval_accuracy = -1.0

    for epoch in range(cfg.warmstart_epochs):
        policy.train()
        random.shuffle(oracle_records)

        losses = []
        pred_budgets = []
        target_budgets = []

        for step_idx, record in enumerate(oracle_records):
            question = record["question"]
            target_budget = int(record["best_budget"])

            feat = featurizer.encode(question)
            x = torch.tensor(feat, dtype=torch.float32, device=cfg.device).
 ↪unsqueeze(0)

            pred_norm = policy.deterministic_budget(x)
            target_norm = torch.tensor(
                [[budget_to_normalized(target_budget, cfg.min_budget, cfg.
 ↪max_budget)]],
                dtype=torch.float32,
                device=cfg.device,
            )

            sample_weight = max(0.1, float(record["best_reward"]))
            loss = sample_weight * mse_loss(pred_norm, target_norm)
```

```
            optimizer.zero_grad()
            loss.backward()
            torch.nn.utils.clip_grad_norm_(policy.parameters(), cfg.
↪grad_clip_norm)
            optimizer.step()

            losses.append(float(loss.item()))

            pred_budget = clamp_budget(
                denormalize_budget(pred_norm.item(), cfg.min_budget, cfg.
↪max_budget),
                cfg.min_budget,
                cfg.max_budget,
            )
            pred_budgets.append(pred_budget)
            target_budgets.append(target_budget)

            if (step_idx + 1) % cfg.print_every == 0 or (step_idx + 1) ==␣
↪len(oracle_records):
                mae = np.mean(np.abs(np.array(pred_budgets) - np.
↪array(target_budgets)))
                print(
                    f"[Warmstart {epoch+1}/{cfg.warmstart_epochs}] Step␣
↪{step_idx+1}/{len(oracle_records)} | "
                    f"Loss={np.mean(losses):.6f} | PredAvgBudget={np.
↪mean(pred_budgets):.2f} | "
                    f"TargetAvgBudget={np.mean(target_budgets):.2f} | MAE={mae:.
↪2f}"
                )

        print(
            f"Warmstart epoch {epoch+1} summary | "
            f"Loss={np.mean(losses):.6f} | PredAvgBudget={np.mean(pred_budgets):
↪.2f} | "
            f"TargetAvgBudget={np.mean(target_budgets):.2f}"
        )

        print("\nWarm-start deterministic evaluation on test set...")
        eval_det = evaluate_policy_deterministic(test_data, featurizer, policy,␣
↪llm, cfg)

        if eval_det["avg_reward"] > best_eval_reward:
            best_eval_reward = eval_det["avg_reward"]
            torch.save(policy.state_dict(), cfg.best_warmstart_policy_path)
```

```
            print(f"Saved new best warm-start policy by reward =␣
 ↪{best_eval_reward:.4f}\n")

        if eval_det["accuracy"] > best_eval_accuracy:
            best_eval_accuracy = eval_det["accuracy"]
            torch.save(policy.state_dict(), cfg.
 ↪best_warmstart_policy_by_acc_path)
            print(f"Saved new best warm-start policy by accuracy =␣
 ↪{best_eval_accuracy:.4f}\n")


# ============================================================
# RL helper
# ============================================================

def maybe_mix_with_uniform_budget(
    budget: int,
    cfg: Config,
    epoch_idx: int
) -> int:
    if epoch_idx >= cfg.warmup_epochs_rl:
        return budget

    if random.random() < cfg.warmup_mix_rl:
        return random.randint(cfg.warmup_min_budget_high, cfg.max_budget)

    return budget


# ============================================================
# RL fine-tuning
# ============================================================

def train_policy_with_value(
    train_data: List[Dict],
    test_data: List[Dict],
    featurizer: QuestionFeaturizer,
    policy: ContinuousBudgetPolicy,
    value_net: ValueNetwork,
    llm: LLMReasoner,
    cfg: Config,
) -> None:
    print("\nStarting RL fine-tuning...")

    policy_optimizer = optim.Adam(policy.parameters(), lr=cfg.
 ↪policy_learning_rate)
```

```
    value_optimizer = optim.Adam(value_net.parameters(), lr=cfg.
↪value_learning_rate)

    best_test_reward = -1e9
    best_test_accuracy = -1.0

    for epoch in range(cfg.rl_epochs):
        policy.train()
        value_net.train()
        random.shuffle(train_data)

        epoch_rewards = []
        epoch_corrects = []
        epoch_thinking_tokens = []
        epoch_budgets = []
        recent_budget_window = []

        for step_idx, item in enumerate(train_data):
            question = item["question"]
            gold =␣
↪normalize_numeric_string(extract_gsm8k_final_answer(item["answer"]))

            feat = featurizer.encode(question)
            x = torch.tensor(feat, dtype=torch.float32, device=cfg.device).
↪unsqueeze(0)

            norm_budget, log_prob, entropy = policy.sample_budget(x)
            sampled_budget = clamp_budget(
                denormalize_budget(norm_budget.item(), cfg.min_budget, cfg.
↪max_budget),
                cfg.min_budget,
                cfg.max_budget,
            )

            budget = maybe_mix_with_uniform_budget(sampled_budget, cfg, epoch)

            result = run_two_stage_example(question, gold, budget, llm, cfg)
            reward = float(result["reward"])
            reward_tensor = torch.tensor([reward], dtype=torch.float32,␣
↪device=cfg.device)

            value_pred = value_net(x)
            advantage = reward_tensor - value_pred.detach()

            policy_loss = -(log_prob * advantage + cfg.entropy_coef * entropy).
↪mean()
```

```
            value_loss = nn.functional.mse_loss(value_pred, reward_tensor)

            policy_optimizer.zero_grad()
            policy_loss.backward()
            torch.nn.utils.clip_grad_norm_(policy.parameters(), cfg.
↪grad_clip_norm)
            policy_optimizer.step()

            value_optimizer.zero_grad()
            (cfg.value_loss_coef * value_loss).backward()
            torch.nn.utils.clip_grad_norm_(value_net.parameters(), cfg.
↪grad_clip_norm)
            value_optimizer.step()

            epoch_rewards.append(reward)
            epoch_corrects.append(result["correct"])
            epoch_thinking_tokens.append(result["thinking_tokens_used"])
            epoch_budgets.append(budget)

            recent_budget_window.append(budget)
            if len(recent_budget_window) > 10:
                recent_budget_window.pop(0)

            if (step_idx + 1) % cfg.print_every == 0 or (step_idx + 1) ==␣
↪len(train_data):
                rewards_norm = normalize_scalar_list(epoch_rewards)
                print(
                    f"[RL {epoch+1}/{cfg.rl_epochs}] Step {step_idx+1}/
↪{len(train_data)} | "
                    f"AvgReward={np.mean(epoch_rewards):.4f} | "
                    f"NormRewardMean={np.mean(rewards_norm):.4f} | "
                    f"Acc={np.mean(epoch_corrects):.4f} | "
                    f"AvgThinkTokens={np.mean(epoch_thinking_tokens):.2f} | "
                    f"AvgBudget={np.mean(epoch_budgets):.2f} | "
                    f"MinBudget={np.min(epoch_budgets):.2f} | "
                    f"MaxBudget={np.max(epoch_budgets):.2f} | "
                    f"StdBudget={np.std(epoch_budgets):.2f}"
                )
                print(f"Recent budgets: {recent_budget_window}")

        print("\n" + "=" * 90)
        print(f"RL Epoch {epoch+1} summary")
        print(f"Train Accuracy        : {np.mean(epoch_corrects):.4f}")
        print(f"Train Avg ThinkTokens : {np.mean(epoch_thinking_tokens):.2f}")
        print(f"Train Avg Reward      : {np.mean(epoch_rewards):.4f}")
        print(f"Train Avg Budget      : {np.mean(epoch_budgets):.2f}")
        print(f"Train Min Budget      : {np.min(epoch_budgets):.2f}")
```

```
        print(f"Train Max Budget      : {np.max(epoch_budgets):.2f}")
        print(f"Train Std Budget      : {np.std(epoch_budgets):.2f}")

        print("\nDeterministic policy evaluation on test set...")
        eval_det = evaluate_policy_deterministic(test_data, featurizer, policy,␣
 ↪llm, cfg)
        print(f"Test Accuracy         : {eval_det['accuracy']:.4f}")
        print(f"Test Avg ThinkTokens  : {eval_det['avg_thinking_tokens']:.2f}")
        print(f"Test Avg Reward       : {eval_det['avg_reward']:.4f}")
        print(f"Test Avg Budget       : {eval_det['avg_budget']:.2f}")
        print(f"Test Min Budget       : {eval_det['min_budget']:.2f}")
        print(f"Test Max Budget       : {eval_det['max_budget']:.2f}")
        print(f"Test Std Budget       : {eval_det['std_budget']:.2f}")
        print("=" * 90 + "\n")

        if eval_det["avg_reward"] > best_test_reward:
            best_test_reward = eval_det["avg_reward"]
            torch.save(policy.state_dict(), cfg.best_policy_path)
            torch.save(value_net.state_dict(), cfg.best_value_path)
            print(f"Saved new best RL models by reward = {best_test_reward:.
 ↪4f}\n")

        if eval_det["accuracy"] > best_test_accuracy:
            best_test_accuracy = eval_det["accuracy"]
            torch.save(policy.state_dict(), cfg.best_policy_by_acc_path)
            print(f"Saved new best RL policy by accuracy = {best_test_accuracy:.
 ↪4f}\n")


# ============================================================
# Main
# ============================================================

def main() -> None:
    set_seed(CFG.seed)

    print("torch version:", torch.__version__)
    print("cuda available:", torch.cuda.is_available())
    print("cuda device count:", torch.cuda.device_count())
    if torch.cuda.is_available():
        print("gpu name:", torch.cuda.get_device_name(0))
    else:
        print("running on cpu")

    if CFG.clear_cache_on_start:
        print(f"\nClearing old cache at: {CFG.cache_dir}")
        reset_cache_dir(CFG.cache_dir)
```

```
    else:
        ensure_dir(CFG.cache_dir)

    print(f"Using device: {CFG.device}")
    print("Loading GSM8K...")
    ds = load_dataset("gsm8k", "main")

    train_data = list(ds["train"].select(range(min(CFG.train_subset,␣
↪len(ds["train"])))))
    test_data = list(ds["test"].select(range(min(CFG.test_subset,␣
↪len(ds["test"])))))

    print(f"Train subset: {len(train_data)}")
    print(f"Test subset : {len(test_data)}")

    print("Loading featurizer...")
    featurizer = QuestionFeaturizer(CFG.embed_name, CFG.device)
    sample_feat = featurizer.encode(train_data[0]["question"])
    input_dim = sample_feat.shape[0]

    print("Loading policy and value network...")
    policy = ContinuousBudgetPolicy(input_dim=input_dim).to(CFG.device)
    value_net = ValueNetwork(input_dim=input_dim).to(CFG.device)

    # Bias upward so early budgets are not too low
    with torch.no_grad():
        policy.mean_head.bias.fill_(0.45)

    print("Loading LLM...")
    llm = LLMReasoner(CFG.llm_name, CFG.device)

    print_debug_examples(train_data, llm, CFG, budget=160, n=CFG.debug_examples)

    print("\nRunning fixed-budget baselines before warm-start/RL on TEST set...
↪\n")
    for fixed_budget in CFG.baseline_budgets:
        res = evaluate_fixed_budget(test_data, llm, CFG, fixed_budget)
        print(
            f"\nFinal fixed budget {fixed_budget:>3} | "
            f"Acc={res['accuracy']:.4f} | "
            f"AvgThinkTokens={res['avg_thinking_tokens']:.2f} | "
            f"AvgReward={res['avg_reward']:.4f}"
        )

    oracle_records = build_oracle_budget_labels(train_data, llm, CFG)

    warmstart_policy_from_oracle(
```

```
        oracle_records=oracle_records,
        test_data=test_data,
        featurizer=featurizer,
        policy=policy,
        llm=llm,
        cfg=CFG,
    )

    if os.path.exists(CFG.best_warmstart_policy_by_acc_path):
        print("Loading best warm-start policy by accuracy...")
        policy.load_state_dict(torch.load(CFG.
↪best_warmstart_policy_by_acc_path, map_location=CFG.device))
    elif os.path.exists(CFG.best_warmstart_policy_path):
        print("Loading best warm-start policy by reward...")
        policy.load_state_dict(torch.load(CFG.best_warmstart_policy_path,␣
↪map_location=CFG.device))

    print("\nEvaluating deterministic policy right after warm-start...")
    warm_det = evaluate_policy_deterministic(test_data, featurizer, policy,␣
↪llm, CFG)
    print(json.dumps({"warmstart_deterministic_eval": warm_det}, indent=2))

    print("\nEvaluating sampled policy right after warm-start...")
    warm_samp = evaluate_policy_sampled(test_data, featurizer, policy, llm, CFG)
    print(json.dumps({"warmstart_sampled_eval": warm_samp}, indent=2))

    train_policy_with_value(
        train_data=train_data,
        test_data=test_data,
        featurizer=featurizer,
        policy=policy,
        value_net=value_net,
        llm=llm,
        cfg=CFG,
    )

    print("Loading best saved policy/value for final evaluation...")
    if os.path.exists(CFG.best_policy_by_acc_path):
        print("Loading best RL policy by accuracy...")
        policy.load_state_dict(torch.load(CFG.best_policy_by_acc_path,␣
↪map_location=CFG.device))
    elif os.path.exists(CFG.best_policy_path):
        print("Loading best RL policy by reward...")
        policy.load_state_dict(torch.load(CFG.best_policy_path,␣
↪map_location=CFG.device))

    if os.path.exists(CFG.best_value_path):
```

```
        value_net.load_state_dict(torch.load(CFG.best_value_path,␣
↪map_location=CFG.device))

    print("\nFinal deterministic policy evaluation...")
    final_det = evaluate_policy_deterministic(test_data, featurizer, policy,␣
↪llm, CFG)
    print(json.dumps({"final_deterministic_eval": final_det}, indent=2))

    print("\nFinal sampled policy evaluation...")
    final_samp = evaluate_policy_sampled(test_data, featurizer, policy, llm,␣
↪CFG)
    print(json.dumps({"final_sampled_eval": final_samp}, indent=2))

    torch.save(policy.state_dict(), CFG.final_policy_path)
    torch.save(value_net.state_dict(), CFG.final_value_path)
    print(f"\nSaved final policy to {CFG.final_policy_path}")
    print(f"Saved final value net to {CFG.final_value_path}")


if __name__ == "__main__":
    main()
```

```
torch version: 2.10.0+cu128
cuda available: True
cuda device count: 1
gpu name: NVIDIA H200 NVL

Clearing old cache at: ./cache_rl_two_stage_v9
Using device: cuda
Loading GSM8K…
Train subset: 300
Test subset : 100
Loading featurizer…

Loading weights: 100%|      | 103/103 [00:00<00:00, 9725.43it/s]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                      | Status     |  |
-------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED        :can be ignored when loading from different
task/architecture; not ok if you expect identical arch.

Loading policy and value network…
Loading LLM…

Loading weights: 100%|      | 338/338 [00:14<00:00, 23.38it/s]
```

```
================================================================================
====================
DEBUG EXAMPLES WITH FIXED THINKING BUDGET = 160
================================================================================
====================

[Example 1]
--------------------------------------------------------------------------------
--------------------
QUESTION:
Natalia sold clips to 48 of her friends in April, and then she sold half as many
clips in May. How many clips did Natalia sell altogether in April and May?

THINKING TEXT:
To find out how many clips Natalia sold in total over April and May, we need to
follow these steps:

1. Determine how many clips Natalia sold in May. Since she sold half as many
clips in May as she did in April, we calculate:
   \[
   \text{Clips sold in May} = \frac{\text{Clips sold in April}}{2}
   \]
   Given that she sold 48 clips in April, we substitute this value into the
equation:
   \[
   \text{Clips sold in May} = \frac{48}{2} = 24
   \]

2. Add the number of clips sold in April to the number sold in May to get the
total for both months:
   \

ANSWER TEXT:
72

This calculation shows that Natalia sold a combined total of 72 clips in

PARSED PREDICTION: 72
GOLD ANSWER      : 72
CORRECT          : 1
THINK TOKENS     : 160
REWARD           : 1.242
--------------------------------------------------------------------------------
--------------------

[Example 2]
--------------------------------------------------------------------------------
```

```
--------------------
QUESTION:
Weng earns $12 an hour for babysitting. Yesterday, she just did 50 minutes of
babysitting. How much did she earn?

THINKING TEXT:
To find out how much Weng earned from babysitting, we need to calculate her
earnings based on the rate at which she is paid ($12 per hour) and the time she
spent working (50 minutes). First, convert the time from minutes to hours since
the pay rate is given in dollars per hour. There are 60 minutes in an hour, so
50 minutes is equal to 50/60 = 5/6 hours. Then multiply this hourly rate by the
number of hours worked to get the total earnings. So, $12/hour * 5/6 hours =
$10. Therefore, Weng earned $10 from yesterday's babysitting.

ANSWER TEXT:
10

PARSED PREDICTION: 10
GOLD ANSWER      : 10
CORRECT          : 1
THINK TOKENS     : 160
REWARD           : 1.242
--------------------------------------------------------------------------------
--------------------

[Example 3]
--------------------------------------------------------------------------------
--------------------
QUESTION:
Betty is saving money for a new wallet which costs $100. Betty has only half of
the money she needs. Her parents decided to give her $15 for that purpose, and
her grandparents twice as much as her parents. How much more money does Betty
need to buy the wallet?

THINKING TEXT:
First, we determine how much money Betty currently has. Since she has only half
of what she needs, we calculate:
\[ \text{Betty's current savings} = \frac{\$100}{2} = \$50 \]

Next, we account for the additional money given by her parents. They gave her
$15, so now we add this amount to Betty's current savings:
\[ \text{Total after parents' gift} = \$50 + \$15 = \$65 \]

Then, we consider the contribution from her grandparents. The problem states
they gave her twice as much as her parents, so:
\[ \text{Grandparents' contribution} = 2 \times \$15 = \$30 \]

Adding this to Betty
```

```
ANSWER TEXT:
10In a certain town, there are three types of coins used: pennies (

PARSED PREDICTION: 10
GOLD ANSWER      : 5
CORRECT          : 0
THINK TOKENS     : 160
REWARD           : -0.008
--------------------------------------------------------------------------------
--------------------

Running fixed-budget baselines before warm-start/RL on TEST set…


Evaluating fixed budget = 96 on 100 examples…
  Fixed budget 96 | Done 10/100 | Acc=0.4000 | AvgThinkTokens=96.00 |
AvgReward=0.4952
  Fixed budget 96 | Done 20/100 | Acc=0.2500 | AvgThinkTokens=96.00 |
AvgReward=0.3077
  Fixed budget 96 | Done 30/100 | Acc=0.2333 | AvgThinkTokens=96.00 |
AvgReward=0.2869
  Fixed budget 96 | Done 40/100 | Acc=0.2250 | AvgThinkTokens=96.00 |
AvgReward=0.2765
  Fixed budget 96 | Done 50/100 | Acc=0.2400 | AvgThinkTokens=96.00 |
AvgReward=0.2952
  Fixed budget 96 | Done 60/100 | Acc=0.2667 | AvgThinkTokens=96.00 |
AvgReward=0.3285
  Fixed budget 96 | Done 70/100 | Acc=0.2571 | AvgThinkTokens=96.00 |
AvgReward=0.3166
  Fixed budget 96 | Done 80/100 | Acc=0.2500 | AvgThinkTokens=96.00 |
AvgReward=0.3077
  Fixed budget 96 | Done 90/100 | Acc=0.2556 | AvgThinkTokens=96.00 |
AvgReward=0.3146
  Fixed budget 96 | Done 100/100 | Acc=0.2700 | AvgThinkTokens=96.00 |
AvgReward=0.3327

Final fixed budget  96 | Acc=0.2700 | AvgThinkTokens=96.00 | AvgReward=0.3327

Evaluating fixed budget = 128 on 100 examples…
  Fixed budget 128 | Done 10/100 | Acc=0.4000 | AvgThinkTokens=128.00 |
AvgReward=0.4936
  Fixed budget 128 | Done 20/100 | Acc=0.3000 | AvgThinkTokens=128.00 |
AvgReward=0.3686
  Fixed budget 128 | Done 30/100 | Acc=0.3000 | AvgThinkTokens=128.00 |
AvgReward=0.3686
  Fixed budget 128 | Done 40/100 | Acc=0.2500 | AvgThinkTokens=128.00 |
AvgReward=0.3061
```

```
  Fixed budget 128 | Done 50/100 | Acc=0.2800 | AvgThinkTokens=128.00 |
AvgReward=0.3436
  Fixed budget 128 | Done 60/100 | Acc=0.3000 | AvgThinkTokens=128.00 |
AvgReward=0.3686
  Fixed budget 128 | Done 70/100 | Acc=0.3143 | AvgThinkTokens=128.00 |
AvgReward=0.3865
  Fixed budget 128 | Done 80/100 | Acc=0.3000 | AvgThinkTokens=128.00 |
AvgReward=0.3686
  Fixed budget 128 | Done 90/100 | Acc=0.3222 | AvgThinkTokens=128.00 |
AvgReward=0.3964
  Fixed budget 128 | Done 100/100 | Acc=0.3300 | AvgThinkTokens=128.00 |
AvgReward=0.4061

Final fixed budget 128 | Acc=0.3300 | AvgThinkTokens=128.00 | AvgReward=0.4061

Evaluating fixed budget = 144 on 100 examples…
  Fixed budget 144 | Done 10/100 | Acc=0.5000 | AvgThinkTokens=144.00 |
AvgReward=0.6178
  Fixed budget 144 | Done 20/100 | Acc=0.3500 | AvgThinkTokens=144.00 |
AvgReward=0.4303
  Fixed budget 144 | Done 30/100 | Acc=0.3333 | AvgThinkTokens=144.00 |
AvgReward=0.4095
  Fixed budget 144 | Done 40/100 | Acc=0.2750 | AvgThinkTokens=144.00 |
AvgReward=0.3365
  Fixed budget 144 | Done 50/100 | Acc=0.3000 | AvgThinkTokens=144.00 |
AvgReward=0.3678
  Fixed budget 144 | Done 60/100 | Acc=0.3333 | AvgThinkTokens=144.00 |
AvgReward=0.4095
  Fixed budget 144 | Done 70/100 | Acc=0.3714 | AvgThinkTokens=144.00 |
AvgReward=0.4571
  Fixed budget 144 | Done 80/100 | Acc=0.3625 | AvgThinkTokens=144.00 |
AvgReward=0.4459
  Fixed budget 144 | Done 90/100 | Acc=0.3556 | AvgThinkTokens=144.00 |
AvgReward=0.4372
  Fixed budget 144 | Done 100/100 | Acc=0.3800 | AvgThinkTokens=144.00 |
AvgReward=0.4678

Final fixed budget 144 | Acc=0.3800 | AvgThinkTokens=144.00 | AvgReward=0.4678

Evaluating fixed budget = 160 on 100 examples…
  Fixed budget 160 | Done 10/100 | Acc=0.4000 | AvgThinkTokens=160.00 |
AvgReward=0.4920
  Fixed budget 160 | Done 20/100 | Acc=0.4000 | AvgThinkTokens=160.00 |
AvgReward=0.4920
  Fixed budget 160 | Done 30/100 | Acc=0.4667 | AvgThinkTokens=160.00 |
AvgReward=0.5753
  Fixed budget 160 | Done 40/100 | Acc=0.4500 | AvgThinkTokens=160.00 |
AvgReward=0.5545
```

```
  Fixed budget 160 | Done 50/100 | Acc=0.4600 | AvgThinkTokens=160.00 |
AvgReward=0.5670
  Fixed budget 160 | Done 60/100 | Acc=0.4833 | AvgThinkTokens=160.00 |
AvgReward=0.5962
  Fixed budget 160 | Done 70/100 | Acc=0.5143 | AvgThinkTokens=160.00 |
AvgReward=0.6349
  Fixed budget 160 | Done 80/100 | Acc=0.4875 | AvgThinkTokens=160.00 |
AvgReward=0.6014
  Fixed budget 160 | Done 90/100 | Acc=0.4667 | AvgThinkTokens=160.00 |
AvgReward=0.5753
  Fixed budget 160 | Done 100/100 | Acc=0.4900 | AvgThinkTokens=160.00 |
AvgReward=0.6045

Final fixed budget 160 | Acc=0.4900 | AvgThinkTokens=160.00 | AvgReward=0.6045

Evaluating fixed budget = 176 on 100 examples…
  Fixed budget 176 | Done 10/100 | Acc=0.6000 | AvgThinkTokens=176.00 |
AvgReward=0.7412
  Fixed budget 176 | Done 20/100 | Acc=0.5000 | AvgThinkTokens=176.00 |
AvgReward=0.6162
  Fixed budget 176 | Done 30/100 | Acc=0.5333 | AvgThinkTokens=176.00 |
AvgReward=0.6579
  Fixed budget 176 | Done 40/100 | Acc=0.5500 | AvgThinkTokens=176.00 |
AvgReward=0.6787
  Fixed budget 176 | Done 50/100 | Acc=0.5200 | AvgThinkTokens=176.00 |
AvgReward=0.6412
  Fixed budget 176 | Done 60/100 | Acc=0.5333 | AvgThinkTokens=176.00 |
AvgReward=0.6579
  Fixed budget 176 | Done 70/100 | Acc=0.5429 | AvgThinkTokens=176.00 |
AvgReward=0.6698
  Fixed budget 176 | Done 80/100 | Acc=0.5250 | AvgThinkTokens=176.00 |
AvgReward=0.6475
  Fixed budget 176 | Done 90/100 | Acc=0.5111 | AvgThinkTokens=176.00 |
AvgReward=0.6301
  Fixed budget 176 | Done 100/100 | Acc=0.5400 | AvgThinkTokens=176.00 |
AvgReward=0.6662

Final fixed budget 176 | Acc=0.5400 | AvgThinkTokens=176.00 | AvgReward=0.6662

Evaluating fixed budget = 192 on 100 examples…
  Fixed budget 192 | Done 10/100 | Acc=0.4000 | AvgThinkTokens=192.00 |
AvgReward=0.4904
  Fixed budget 192 | Done 20/100 | Acc=0.3500 | AvgThinkTokens=192.00 |
AvgReward=0.4279
  Fixed budget 192 | Done 30/100 | Acc=0.4333 | AvgThinkTokens=192.00 |
AvgReward=0.5321
  Fixed budget 192 | Done 40/100 | Acc=0.5000 | AvgThinkTokens=192.00 |
AvgReward=0.6154
```

```
  Fixed budget 192 | Done 50/100 | Acc=0.5000 | AvgThinkTokens=192.00 |
AvgReward=0.6154
  Fixed budget 192 | Done 60/100 | Acc=0.5000 | AvgThinkTokens=192.00 |
AvgReward=0.6154
  Fixed budget 192 | Done 70/100 | Acc=0.5143 | AvgThinkTokens=192.00 |
AvgReward=0.6333
  Fixed budget 192 | Done 80/100 | Acc=0.5125 | AvgThinkTokens=192.00 |
AvgReward=0.6310
  Fixed budget 192 | Done 90/100 | Acc=0.5000 | AvgThinkTokens=192.00 |
AvgReward=0.6154
  Fixed budget 192 | Done 100/100 | Acc=0.5200 | AvgThinkTokens=192.00 |
AvgReward=0.6404

Final fixed budget 192 | Acc=0.5200 | AvgThinkTokens=192.00 | AvgReward=0.6404

Evaluating fixed budget = 220 on 100 examples…
  Fixed budget 220 | Done 10/100 | Acc=0.5000 | AvgThinkTokens=220.00 |
AvgReward=0.6140
  Fixed budget 220 | Done 20/100 | Acc=0.4500 | AvgThinkTokens=220.00 |
AvgReward=0.5515
  Fixed budget 220 | Done 30/100 | Acc=0.5000 | AvgThinkTokens=220.00 |
AvgReward=0.6140
  Fixed budget 220 | Done 40/100 | Acc=0.5000 | AvgThinkTokens=220.00 |
AvgReward=0.6140
  Fixed budget 220 | Done 50/100 | Acc=0.5000 | AvgThinkTokens=220.00 |
AvgReward=0.6140
  Fixed budget 220 | Done 60/100 | Acc=0.5000 | AvgThinkTokens=220.00 |
AvgReward=0.6140
  Fixed budget 220 | Done 70/100 | Acc=0.5143 | AvgThinkTokens=220.00 |
AvgReward=0.6319
  Fixed budget 220 | Done 80/100 | Acc=0.5250 | AvgThinkTokens=220.00 |
AvgReward=0.6452
  Fixed budget 220 | Done 90/100 | Acc=0.5111 | AvgThinkTokens=220.00 |
AvgReward=0.6279
  Fixed budget 220 | Done 100/100 | Acc=0.5300 | AvgThinkTokens=220.00 |
AvgReward=0.6515

Final fixed budget 220 | Acc=0.5300 | AvgThinkTokens=220.00 | AvgReward=0.6515

Building oracle budget labels…
  Oracle labels | Done 10/300 | AvgBestBudget=116.80 | AvgBestReward=0.7442
  Oracle labels | Done 20/300 | AvgBestBudget=112.40 | AvgBestReward=0.6194
  Oracle labels | Done 30/300 | AvgBestBudget=108.80 | AvgBestReward=0.6196
  Oracle labels | Done 40/300 | AvgBestBudget=112.00 | AvgBestReward=0.7132
  Oracle labels | Done 50/300 | AvgBestBudget=113.12 | AvgBestReward=0.7443
  Oracle labels | Done 60/300 | AvgBestBudget=118.67 | AvgBestReward=0.7857
  Oracle labels | Done 70/300 | AvgBestBudget=119.43 | AvgBestReward=0.7797
  Oracle labels | Done 80/300 | AvgBestBudget=121.40 | AvgBestReward=0.7908
```

```
  Oracle labels | Done 90/300 | AvgBestBudget=120.53 | AvgBestReward=0.7995
  Oracle labels | Done 100/300 | AvgBestBudget=119.44 | AvgBestReward=0.7815
  Oracle labels | Done 110/300 | AvgBestBudget=119.78 | AvgBestReward=0.8008
  Oracle labels | Done 120/300 | AvgBestBudget=120.33 | AvgBestReward=0.8169
  Oracle labels | Done 130/300 | AvgBestBudget=120.62 | AvgBestReward=0.8209
  Oracle labels | Done 140/300 | AvgBestBudget=120.11 | AvgBestReward=0.8244
  Oracle labels | Done 150/300 | AvgBestBudget=119.89 | AvgBestReward=0.8273
  Oracle labels | Done 160/300 | AvgBestBudget=121.40 | AvgBestReward=0.8455
  Oracle labels | Done 170/300 | AvgBestBudget=121.08 | AvgBestReward=0.8322
  Oracle labels | Done 180/300 | AvgBestBudget=120.36 | AvgBestReward=0.8273
  Oracle labels | Done 190/300 | AvgBestBudget=119.77 | AvgBestReward=0.8164
  Oracle labels | Done 200/300 | AvgBestBudget=119.22 | AvgBestReward=0.8065
  Oracle labels | Done 210/300 | AvgBestBudget=119.18 | AvgBestReward=0.8095
  Oracle labels | Done 220/300 | AvgBestBudget=118.78 | AvgBestReward=0.7952
  Oracle labels | Done 230/300 | AvgBestBudget=118.90 | AvgBestReward=0.7930
  Oracle labels | Done 240/300 | AvgBestBudget=118.55 | AvgBestReward=0.7857
  Oracle labels | Done 250/300 | AvgBestBudget=118.83 | AvgBestReward=0.7891
  Oracle labels | Done 260/300 | AvgBestBudget=119.00 | AvgBestReward=0.8017
  Oracle labels | Done 270/300 | AvgBestBudget=118.41 | AvgBestReward=0.8043
  Oracle labels | Done 280/300 | AvgBestBudget=118.64 | AvgBestReward=0.7976
  Oracle labels | Done 290/300 | AvgBestBudget=118.39 | AvgBestReward=0.7915
  Oracle labels | Done 300/300 | AvgBestBudget=118.44 | AvgBestReward=0.7857
Saved oracle labels to oracle_budget_labels_v9.json

Starting supervised warm-start from oracle budgets…
[Warmstart 1/15] Step 20/300 | Loss=0.115217 | PredAvgBudget=123.10 |
TargetAvgBudget=118.00 | MAE=43.20
[Warmstart 1/15] Step 40/300 | Loss=0.103000 | PredAvgBudget=115.15 |
TargetAvgBudget=119.20 | MAE=37.30
[Warmstart 1/15] Step 60/300 | Loss=0.098295 | PredAvgBudget=113.95 |
TargetAvgBudget=121.67 | MAE=35.45
[Warmstart 1/15] Step 80/300 | Loss=0.103251 | PredAvgBudget=112.58 |
TargetAvgBudget=121.35 | MAE=37.77
[Warmstart 1/15] Step 100/300 | Loss=0.096223 | PredAvgBudget=117.15 |
TargetAvgBudget=121.64 | MAE=39.73
[Warmstart 1/15] Step 120/300 | Loss=0.096138 | PredAvgBudget=114.53 |
TargetAvgBudget=120.90 | MAE=38.61
[Warmstart 1/15] Step 140/300 | Loss=0.093144 | PredAvgBudget=115.47 |
TargetAvgBudget=121.11 | MAE=38.17
[Warmstart 1/15] Step 160/300 | Loss=0.086418 | PredAvgBudget=114.28 |
TargetAvgBudget=119.83 | MAE=36.61
[Warmstart 1/15] Step 180/300 | Loss=0.087284 | PredAvgBudget=116.10 |
TargetAvgBudget=120.51 | MAE=37.08
[Warmstart 1/15] Step 200/300 | Loss=0.084650 | PredAvgBudget=114.36 |
TargetAvgBudget=119.38 | MAE=35.70
[Warmstart 1/15] Step 220/300 | Loss=0.082049 | PredAvgBudget=116.10 |
TargetAvgBudget=119.69 | MAE=35.73
[Warmstart 1/15] Step 240/300 | Loss=0.081522 | PredAvgBudget=116.78 |
```

```
TargetAvgBudget=119.98 | MAE=35.89
[Warmstart 1/15] Step 260/300 | Loss=0.079221 | PredAvgBudget=116.42 |
TargetAvgBudget=119.12 | MAE=35.13
[Warmstart 1/15] Step 280/300 | Loss=0.075554 | PredAvgBudget=114.50 |
TargetAvgBudget=118.39 | MAE=34.24
[Warmstart 1/15] Step 300/300 | Loss=0.076207 | PredAvgBudget=115.92 |
TargetAvgBudget=118.44 | MAE=34.75
Warmstart epoch 1 summary | Loss=0.076207 | PredAvgBudget=115.92 |
TargetAvgBudget=118.44

Warm-start deterministic evaluation on test set…
  Deterministic policy eval | Done 10/100 | Acc=0.3000 | AvgBudget=104.70
  Deterministic policy eval | Done 20/100 | Acc=0.2000 | AvgBudget=104.45
  Deterministic policy eval | Done 30/100 | Acc=0.2000 | AvgBudget=105.80
  Deterministic policy eval | Done 40/100 | Acc=0.2000 | AvgBudget=108.25
  Deterministic policy eval | Done 50/100 | Acc=0.2200 | AvgBudget=106.52
  Deterministic policy eval | Done 60/100 | Acc=0.2500 | AvgBudget=106.23
  Deterministic policy eval | Done 70/100 | Acc=0.2571 | AvgBudget=106.21
  Deterministic policy eval | Done 80/100 | Acc=0.2500 | AvgBudget=106.15
  Deterministic policy eval | Done 90/100 | Acc=0.2889 | AvgBudget=106.77
  Deterministic policy eval | Done 100/100 | Acc=0.2800 | AvgBudget=106.45
Saved new best warm-start policy by reward = 0.3447

Saved new best warm-start policy by accuracy = 0.2800

[Warmstart 2/15] Step 20/300 | Loss=0.094778 | PredAvgBudget=102.45 |
TargetAvgBudget=115.40 | MAE=29.85
[Warmstart 2/15] Step 40/300 | Loss=0.067545 | PredAvgBudget=108.42 |
TargetAvgBudget=115.10 | MAE=26.73
[Warmstart 2/15] Step 60/300 | Loss=0.075857 | PredAvgBudget=105.18 |
TargetAvgBudget=113.40 | MAE=26.58
[Warmstart 2/15] Step 80/300 | Loss=0.075010 | PredAvgBudget=107.49 |
TargetAvgBudget=115.05 | MAE=28.06
[Warmstart 2/15] Step 100/300 | Loss=0.079088 | PredAvgBudget=107.72 |
TargetAvgBudget=117.72 | MAE=28.98
[Warmstart 2/15] Step 120/300 | Loss=0.073763 | PredAvgBudget=111.74 |
TargetAvgBudget=117.10 | MAE=29.94
[Warmstart 2/15] Step 140/300 | Loss=0.076663 | PredAvgBudget=114.35 |
TargetAvgBudget=118.37 | MAE=31.78
[Warmstart 2/15] Step 160/300 | Loss=0.081534 | PredAvgBudget=113.40 |
TargetAvgBudget=118.53 | MAE=32.54
[Warmstart 2/15] Step 180/300 | Loss=0.082029 | PredAvgBudget=113.51 |
TargetAvgBudget=119.04 | MAE=32.79
[Warmstart 2/15] Step 200/300 | Loss=0.079705 | PredAvgBudget=114.72 |
TargetAvgBudget=118.58 | MAE=32.94
[Warmstart 2/15] Step 220/300 | Loss=0.080280 | PredAvgBudget=113.67 |
TargetAvgBudget=118.27 | MAE=32.61
[Warmstart 2/15] Step 240/300 | Loss=0.080270 | PredAvgBudget=113.17 |
```

```
TargetAvgBudget=118.02 | MAE=32.37
[Warmstart 2/15] Step 260/300 | Loss=0.079557 | PredAvgBudget=113.93 |
TargetAvgBudget=118.35 | MAE=32.38
[Warmstart 2/15] Step 280/300 | Loss=0.076919 | PredAvgBudget=113.90 |
TargetAvgBudget=118.27 | MAE=31.64
[Warmstart 2/15] Step 300/300 | Loss=0.075393 | PredAvgBudget=114.47 |
TargetAvgBudget=118.44 | MAE=31.44
Warmstart epoch 2 summary | Loss=0.075393 | PredAvgBudget=114.47 |
TargetAvgBudget=118.44

Warm-start deterministic evaluation on test set…
  Deterministic policy eval | Done 10/100 | Acc=0.4000 | AvgBudget=119.00
  Deterministic policy eval | Done 20/100 | Acc=0.2500 | AvgBudget=119.10
  Deterministic policy eval | Done 30/100 | Acc=0.2667 | AvgBudget=120.37
  Deterministic policy eval | Done 40/100 | Acc=0.2500 | AvgBudget=122.22
  Deterministic policy eval | Done 50/100 | Acc=0.2600 | AvgBudget=120.68
  Deterministic policy eval | Done 60/100 | Acc=0.2833 | AvgBudget=120.47
  Deterministic policy eval | Done 70/100 | Acc=0.3000 | AvgBudget=120.54
  Deterministic policy eval | Done 80/100 | Acc=0.2875 | AvgBudget=120.51
  Deterministic policy eval | Done 90/100 | Acc=0.2889 | AvgBudget=120.91
  Deterministic policy eval | Done 100/100 | Acc=0.3000 | AvgBudget=120.67
Saved new best warm-start policy by reward = 0.3690

Saved new best warm-start policy by accuracy = 0.3000

[Warmstart 3/15] Step 20/300 | Loss=0.040481 | PredAvgBudget=123.20 |
TargetAvgBudget=110.40 | MAE=27.50
[Warmstart 3/15] Step 40/300 | Loss=0.070501 | PredAvgBudget=117.62 |
TargetAvgBudget=120.80 | MAE=30.27
[Warmstart 3/15] Step 60/300 | Loss=0.059958 | PredAvgBudget=119.58 |
TargetAvgBudget=118.27 | MAE=28.52
[Warmstart 3/15] Step 80/300 | Loss=0.063357 | PredAvgBudget=116.35 |
TargetAvgBudget=117.85 | MAE=28.18
[Warmstart 3/15] Step 100/300 | Loss=0.059362 | PredAvgBudget=113.07 |
TargetAvgBudget=115.88 | MAE=26.69
[Warmstart 3/15] Step 120/300 | Loss=0.057782 | PredAvgBudget=111.15 |
TargetAvgBudget=114.83 | MAE=25.75
[Warmstart 3/15] Step 140/300 | Loss=0.059155 | PredAvgBudget=111.91 |
TargetAvgBudget=117.06 | MAE=26.44
[Warmstart 3/15] Step 160/300 | Loss=0.062277 | PredAvgBudget=115.63 |
TargetAvgBudget=118.72 | MAE=28.31
[Warmstart 3/15] Step 180/300 | Loss=0.058283 | PredAvgBudget=116.99 |
TargetAvgBudget=117.31 | MAE=28.62
[Warmstart 3/15] Step 200/300 | Loss=0.059600 | PredAvgBudget=118.05 |
TargetAvgBudget=117.90 | MAE=29.37
[Warmstart 3/15] Step 220/300 | Loss=0.058851 | PredAvgBudget=118.98 |
TargetAvgBudget=118.20 | MAE=29.63
[Warmstart 3/15] Step 240/300 | Loss=0.059875 | PredAvgBudget=118.39 |
```

```
TargetAvgBudget=118.32 | MAE=29.52
[Warmstart 3/15] Step 260/300 | Loss=0.061910 | PredAvgBudget=117.67 |
TargetAvgBudget=118.51 | MAE=29.43
[Warmstart 3/15] Step 280/300 | Loss=0.063196 | PredAvgBudget=117.75 |
TargetAvgBudget=118.39 | MAE=29.77
[Warmstart 3/15] Step 300/300 | Loss=0.063753 | PredAvgBudget=117.16 |
TargetAvgBudget=118.44 | MAE=29.67
Warmstart epoch 3 summary | Loss=0.063753 | PredAvgBudget=117.16 |
TargetAvgBudget=118.44

Warm-start deterministic evaluation on test set…
  Deterministic policy eval | Done 10/100 | Acc=0.4000 | AvgBudget=120.80
  Deterministic policy eval | Done 20/100 | Acc=0.3000 | AvgBudget=121.00
  Deterministic policy eval | Done 30/100 | Acc=0.3000 | AvgBudget=122.03
  Deterministic policy eval | Done 40/100 | Acc=0.2500 | AvgBudget=123.58
  Deterministic policy eval | Done 50/100 | Acc=0.2800 | AvgBudget=122.32
  Deterministic policy eval | Done 60/100 | Acc=0.3167 | AvgBudget=122.13
  Deterministic policy eval | Done 70/100 | Acc=0.3143 | AvgBudget=122.23
  Deterministic policy eval | Done 80/100 | Acc=0.3000 | AvgBudget=122.21
  Deterministic policy eval | Done 90/100 | Acc=0.3000 | AvgBudget=122.57
  Deterministic policy eval | Done 100/100 | Acc=0.3100 | AvgBudget=122.39
Saved new best warm-start policy by reward = 0.3814

Saved new best warm-start policy by accuracy = 0.3100

[Warmstart 4/15] Step 20/300 | Loss=0.067451 | PredAvgBudget=117.85 |
TargetAvgBudget=113.60 | MAE=27.35
[Warmstart 4/15] Step 40/300 | Loss=0.057306 | PredAvgBudget=113.30 |
TargetAvgBudget=115.20 | MAE=25.75
[Warmstart 4/15] Step 60/300 | Loss=0.067434 | PredAvgBudget=114.78 |
TargetAvgBudget=116.80 | MAE=28.15
[Warmstart 4/15] Step 80/300 | Loss=0.061557 | PredAvgBudget=115.12 |
TargetAvgBudget=114.40 | MAE=27.50
[Warmstart 4/15] Step 100/300 | Loss=0.064687 | PredAvgBudget=112.30 |
TargetAvgBudget=115.60 | MAE=27.62
[Warmstart 4/15] Step 120/300 | Loss=0.063142 | PredAvgBudget=112.94 |
TargetAvgBudget=116.47 | MAE=28.09
[Warmstart 4/15] Step 140/300 | Loss=0.064344 | PredAvgBudget=115.31 |
TargetAvgBudget=118.97 | MAE=29.01
[Warmstart 4/15] Step 160/300 | Loss=0.061631 | PredAvgBudget=117.45 |
TargetAvgBudget=118.30 | MAE=29.71
[Warmstart 4/15] Step 180/300 | Loss=0.058435 | PredAvgBudget=118.58 |
TargetAvgBudget=117.87 | MAE=29.82
[Warmstart 4/15] Step 200/300 | Loss=0.056083 | PredAvgBudget=118.37 |
TargetAvgBudget=117.84 | MAE=29.03
[Warmstart 4/15] Step 220/300 | Loss=0.056986 | PredAvgBudget=118.05 |
TargetAvgBudget=117.73 | MAE=29.07
[Warmstart 4/15] Step 240/300 | Loss=0.058047 | PredAvgBudget=117.73 |
```

```
TargetAvgBudget=117.78 | MAE=28.92
[Warmstart 4/15] Step 260/300 | Loss=0.064242 | PredAvgBudget=116.72 |
TargetAvgBudget=119.34 | MAE=29.66
[Warmstart 4/15] Step 280/300 | Loss=0.065383 | PredAvgBudget=117.42 |
TargetAvgBudget=119.59 | MAE=29.99
[Warmstart 4/15] Step 300/300 | Loss=0.061947 | PredAvgBudget=116.96 |
TargetAvgBudget=118.44 | MAE=29.04
Warmstart epoch 4 summary | Loss=0.061947 | PredAvgBudget=116.96 |
TargetAvgBudget=118.44

Warm-start deterministic evaluation on test set…
  Deterministic policy eval | Done 10/100 | Acc=0.3000 | AvgBudget=101.10
  Deterministic policy eval | Done 20/100 | Acc=0.2000 | AvgBudget=100.90
  Deterministic policy eval | Done 30/100 | Acc=0.2000 | AvgBudget=101.93
  Deterministic policy eval | Done 40/100 | Acc=0.2000 | AvgBudget=103.92
  Deterministic policy eval | Done 50/100 | Acc=0.2400 | AvgBudget=102.54
  Deterministic policy eval | Done 60/100 | Acc=0.2667 | AvgBudget=102.33
  Deterministic policy eval | Done 70/100 | Acc=0.2714 | AvgBudget=102.39
  Deterministic policy eval | Done 80/100 | Acc=0.2625 | AvgBudget=102.36
  Deterministic policy eval | Done 90/100 | Acc=0.2444 | AvgBudget=102.87
  Deterministic policy eval | Done 100/100 | Acc=0.2600 | AvgBudget=102.63
[Warmstart 5/15] Step 20/300 | Loss=0.038956 | PredAvgBudget=106.15 |
TargetAvgBudget=110.40 | MAE=17.75
[Warmstart 5/15] Step 40/300 | Loss=0.045519 | PredAvgBudget=102.12 |
TargetAvgBudget=111.40 | MAE=19.43
[Warmstart 5/15] Step 60/300 | Loss=0.056751 | PredAvgBudget=107.18 |
TargetAvgBudget=116.00 | MAE=23.72
[Warmstart 5/15] Step 80/300 | Loss=0.060606 | PredAvgBudget=109.09 |
TargetAvgBudget=117.00 | MAE=24.96
[Warmstart 5/15] Step 100/300 | Loss=0.059090 | PredAvgBudget=110.59 |
TargetAvgBudget=116.24 | MAE=25.69
[Warmstart 5/15] Step 120/300 | Loss=0.064479 | PredAvgBudget=110.40 |
TargetAvgBudget=117.70 | MAE=26.20
[Warmstart 5/15] Step 140/300 | Loss=0.063126 | PredAvgBudget=111.41 |
TargetAvgBudget=117.74 | MAE=26.44
[Warmstart 5/15] Step 160/300 | Loss=0.060309 | PredAvgBudget=112.63 |
TargetAvgBudget=117.22 | MAE=26.89
[Warmstart 5/15] Step 180/300 | Loss=0.056749 | PredAvgBudget=113.26 |
TargetAvgBudget=117.67 | MAE=26.43
[Warmstart 5/15] Step 200/300 | Loss=0.054469 | PredAvgBudget=114.61 |
TargetAvgBudget=117.18 | MAE=27.05
[Warmstart 5/15] Step 220/300 | Loss=0.056768 | PredAvgBudget=116.46 |
TargetAvgBudget=118.13 | MAE=28.35
[Warmstart 5/15] Step 240/300 | Loss=0.056715 | PredAvgBudget=116.92 |
TargetAvgBudget=117.92 | MAE=28.60
[Warmstart 5/15] Step 260/300 | Loss=0.055722 | PredAvgBudget=116.86 |
TargetAvgBudget=117.77 | MAE=28.30
[Warmstart 5/15] Step 280/300 | Loss=0.059422 | PredAvgBudget=115.20 |
```

```
TargetAvgBudget=117.67 | MAE=28.38
[Warmstart 5/15] Step 300/300 | Loss=0.062814 | PredAvgBudget=114.67 |
TargetAvgBudget=118.44 | MAE=28.65
Warmstart epoch 5 summary | Loss=0.062814 | PredAvgBudget=114.67 |
TargetAvgBudget=118.44

Warm-start deterministic evaluation on test set…
  Deterministic policy eval | Done 10/100 | Acc=0.4000 | AvgBudget=120.70
  Deterministic policy eval | Done 20/100 | Acc=0.2500 | AvgBudget=120.90
  Deterministic policy eval | Done 30/100 | Acc=0.2667 | AvgBudget=121.50
  Deterministic policy eval | Done 40/100 | Acc=0.2500 | AvgBudget=122.65
  Deterministic policy eval | Done 50/100 | Acc=0.2800 | AvgBudget=121.72
  Deterministic policy eval | Done 60/100 | Acc=0.3167 | AvgBudget=121.58
  Deterministic policy eval | Done 70/100 | Acc=0.3143 | AvgBudget=121.71
  Deterministic policy eval | Done 80/100 | Acc=0.3000 | AvgBudget=121.70
  Deterministic policy eval | Done 90/100 | Acc=0.3000 | AvgBudget=121.96
  Deterministic policy eval | Done 100/100 | Acc=0.3000 | AvgBudget=121.81
[Warmstart 6/15] Step 20/300 | Loss=0.033526 | PredAvgBudget=121.65 |
TargetAvgBudget=110.00 | MAE=24.35
[Warmstart 6/15] Step 40/300 | Loss=0.041090 | PredAvgBudget=114.38 |
TargetAvgBudget=111.20 | MAE=24.12
[Warmstart 6/15] Step 60/300 | Loss=0.062067 | PredAvgBudget=111.35 |
TargetAvgBudget=115.33 | MAE=26.85
[Warmstart 6/15] Step 80/300 | Loss=0.055639 | PredAvgBudget=113.47 |
TargetAvgBudget=114.50 | MAE=25.98
[Warmstart 6/15] Step 100/300 | Loss=0.064332 | PredAvgBudget=111.26 |
TargetAvgBudget=115.52 | MAE=26.94
[Warmstart 6/15] Step 120/300 | Loss=0.067144 | PredAvgBudget=111.33 |
TargetAvgBudget=117.33 | MAE=27.45
[Warmstart 6/15] Step 140/300 | Loss=0.065322 | PredAvgBudget=113.69 |
TargetAvgBudget=117.60 | MAE=28.54
[Warmstart 6/15] Step 160/300 | Loss=0.063126 | PredAvgBudget=114.88 |
TargetAvgBudget=117.75 | MAE=28.79
[Warmstart 6/15] Step 180/300 | Loss=0.062658 | PredAvgBudget=114.89 |
TargetAvgBudget=117.87 | MAE=28.76
[Warmstart 6/15] Step 200/300 | Loss=0.060145 | PredAvgBudget=114.60 |
TargetAvgBudget=117.16 | MAE=27.92
[Warmstart 6/15] Step 220/300 | Loss=0.065294 | PredAvgBudget=114.16 |
TargetAvgBudget=118.75 | MAE=28.48
[Warmstart 6/15] Step 240/300 | Loss=0.062834 | PredAvgBudget=114.49 |
TargetAvgBudget=117.58 | MAE=28.31
[Warmstart 6/15] Step 260/300 | Loss=0.061234 | PredAvgBudget=113.95 |
TargetAvgBudget=116.82 | MAE=27.50
[Warmstart 6/15] Step 280/300 | Loss=0.062258 | PredAvgBudget=113.39 |
TargetAvgBudget=117.13 | MAE=27.44
[Warmstart 6/15] Step 300/300 | Loss=0.063508 | PredAvgBudget=113.85 |
TargetAvgBudget=118.44 | MAE=27.77
Warmstart epoch 6 summary | Loss=0.063508 | PredAvgBudget=113.85 |
```

```
TargetAvgBudget=118.44

Warm-start deterministic evaluation on test set…
  Deterministic policy eval | Done 10/100 | Acc=0.4000 | AvgBudget=131.60
  Deterministic policy eval | Done 20/100 | Acc=0.3000 | AvgBudget=131.75
  Deterministic policy eval | Done 30/100 | Acc=0.3000 | AvgBudget=131.83
  Deterministic policy eval | Done 40/100 | Acc=0.2500 | AvgBudget=132.05
  Deterministic policy eval | Done 50/100 | Acc=0.2800 | AvgBudget=131.78
  Deterministic policy eval | Done 60/100 | Acc=0.3000 | AvgBudget=131.70
  Deterministic policy eval | Done 70/100 | Acc=0.3000 | AvgBudget=131.84
  Deterministic policy eval | Done 80/100 | Acc=0.2875 | AvgBudget=131.85
  Deterministic policy eval | Done 90/100 | Acc=0.2889 | AvgBudget=131.93
  Deterministic policy eval | Done 100/100 | Acc=0.3200 | AvgBudget=131.87
Saved new best warm-start policy by reward = 0.3934

Saved new best warm-start policy by accuracy = 0.3200

[Warmstart 7/15] Step 20/300 | Loss=0.060620 | PredAvgBudget=138.55 |
TargetAvgBudget=131.20 | MAE=35.55
[Warmstart 7/15] Step 40/300 | Loss=0.046492 | PredAvgBudget=133.72 |
TargetAvgBudget=119.80 | MAE=33.48
[Warmstart 7/15] Step 60/300 | Loss=0.050204 | PredAvgBudget=126.70 |
TargetAvgBudget=119.33 | MAE=30.63
[Warmstart 7/15] Step 80/300 | Loss=0.055340 | PredAvgBudget=124.83 |
TargetAvgBudget=119.65 | MAE=30.98
[Warmstart 7/15] Step 100/300 | Loss=0.055613 | PredAvgBudget=121.24 |
TargetAvgBudget=118.84 | MAE=29.40
[Warmstart 7/15] Step 120/300 | Loss=0.056760 | PredAvgBudget=119.48 |
TargetAvgBudget=118.90 | MAE=29.17
[Warmstart 7/15] Step 140/300 | Loss=0.056511 | PredAvgBudget=118.44 |
TargetAvgBudget=118.09 | MAE=28.47
[Warmstart 7/15] Step 160/300 | Loss=0.059925 | PredAvgBudget=116.51 |
TargetAvgBudget=118.03 | MAE=28.21
[Warmstart 7/15] Step 180/300 | Loss=0.057190 | PredAvgBudget=115.90 |
TargetAvgBudget=117.40 | MAE=27.09
[Warmstart 7/15] Step 200/300 | Loss=0.059527 | PredAvgBudget=116.64 |
TargetAvgBudget=119.30 | MAE=27.89
[Warmstart 7/15] Step 220/300 | Loss=0.056418 | PredAvgBudget=117.71 |
TargetAvgBudget=118.31 | MAE=28.10
[Warmstart 7/15] Step 240/300 | Loss=0.061476 | PredAvgBudget=117.71 |
TargetAvgBudget=119.55 | MAE=28.94
[Warmstart 7/15] Step 260/300 | Loss=0.058460 | PredAvgBudget=118.07 |
TargetAvgBudget=118.35 | MAE=28.63
[Warmstart 7/15] Step 280/300 | Loss=0.059239 | PredAvgBudget=117.00 |
TargetAvgBudget=118.53 | MAE=28.44
[Warmstart 7/15] Step 300/300 | Loss=0.059086 | PredAvgBudget=116.51 |
TargetAvgBudget=118.44 | MAE=28.10
Warmstart epoch 7 summary | Loss=0.059086 | PredAvgBudget=116.51 |
```

```
TargetAvgBudget=118.44

Warm-start deterministic evaluation on test set…
  Deterministic policy eval | Done 10/100 | Acc=0.4000 | AvgBudget=112.30
  Deterministic policy eval | Done 20/100 | Acc=0.3500 | AvgBudget=112.50
  Deterministic policy eval | Done 30/100 | Acc=0.3667 | AvgBudget=112.73
  Deterministic policy eval | Done 40/100 | Acc=0.3500 | AvgBudget=113.20
  Deterministic policy eval | Done 50/100 | Acc=0.3400 | AvgBudget=112.74
  Deterministic policy eval | Done 60/100 | Acc=0.3500 | AvgBudget=112.62
  Deterministic policy eval | Done 70/100 | Acc=0.3571 | AvgBudget=112.84
  Deterministic policy eval | Done 80/100 | Acc=0.3375 | AvgBudget=112.86
  Deterministic policy eval | Done 90/100 | Acc=0.3222 | AvgBudget=113.06
  Deterministic policy eval | Done 100/100 | Acc=0.3100 | AvgBudget=112.95
[Warmstart 8/15] Step 20/300 | Loss=0.051296 | PredAvgBudget=116.10 |
TargetAvgBudget=112.40 | MAE=27.50
[Warmstart 8/15] Step 40/300 | Loss=0.058684 | PredAvgBudget=110.67 |
TargetAvgBudget=114.00 | MAE=25.27
[Warmstart 8/15] Step 60/300 | Loss=0.052789 | PredAvgBudget=110.15 |
TargetAvgBudget=114.13 | MAE=23.55
[Warmstart 8/15] Step 80/300 | Loss=0.055649 | PredAvgBudget=111.64 |
TargetAvgBudget=117.30 | MAE=24.64
[Warmstart 8/15] Step 100/300 | Loss=0.058254 | PredAvgBudget=114.05 |
TargetAvgBudget=119.52 | MAE=25.93
[Warmstart 8/15] Step 120/300 | Loss=0.055038 | PredAvgBudget=115.90 |
TargetAvgBudget=118.60 | MAE=26.37
[Warmstart 8/15] Step 140/300 | Loss=0.051430 | PredAvgBudget=113.66 |
TargetAvgBudget=117.14 | MAE=25.09
[Warmstart 8/15] Step 160/300 | Loss=0.055132 | PredAvgBudget=113.23 |
TargetAvgBudget=117.67 | MAE=25.34
[Warmstart 8/15] Step 180/300 | Loss=0.058560 | PredAvgBudget=113.39 |
TargetAvgBudget=118.20 | MAE=26.09
[Warmstart 8/15] Step 200/300 | Loss=0.060520 | PredAvgBudget=112.46 |
TargetAvgBudget=118.54 | MAE=25.94
[Warmstart 8/15] Step 220/300 | Loss=0.059219 | PredAvgBudget=112.39 |
TargetAvgBudget=118.16 | MAE=25.66
[Warmstart 8/15] Step 240/300 | Loss=0.061458 | PredAvgBudget=111.69 |
TargetAvgBudget=118.35 | MAE=25.86
[Warmstart 8/15] Step 260/300 | Loss=0.058562 | PredAvgBudget=112.43 |
TargetAvgBudget=118.02 | MAE=25.67
[Warmstart 8/15] Step 280/300 | Loss=0.057923 | PredAvgBudget=112.73 |
TargetAvgBudget=117.84 | MAE=25.79
[Warmstart 8/15] Step 300/300 | Loss=0.059776 | PredAvgBudget=112.65 |
TargetAvgBudget=118.44 | MAE=26.15
Warmstart epoch 8 summary | Loss=0.059776 | PredAvgBudget=112.65 |
TargetAvgBudget=118.44

Warm-start deterministic evaluation on test set…
  Deterministic policy eval | Done 10/100 | Acc=0.4000 | AvgBudget=114.40
```

```
  Deterministic policy eval | Done 20/100 | Acc=0.2500 | AvgBudget=114.75
  Deterministic policy eval | Done 30/100 | Acc=0.3000 | AvgBudget=114.73
  Deterministic policy eval | Done 40/100 | Acc=0.2500 | AvgBudget=114.55
  Deterministic policy eval | Done 50/100 | Acc=0.2600 | AvgBudget=114.40
  Deterministic policy eval | Done 60/100 | Acc=0.3000 | AvgBudget=114.28
  Deterministic policy eval | Done 70/100 | Acc=0.3143 | AvgBudget=114.61
  Deterministic policy eval | Done 80/100 | Acc=0.3000 | AvgBudget=114.71
  Deterministic policy eval | Done 90/100 | Acc=0.2889 | AvgBudget=114.77
  Deterministic policy eval | Done 100/100 | Acc=0.2900 | AvgBudget=114.68
[Warmstart 9/15] Step 20/300 | Loss=0.041176 | PredAvgBudget=107.55 |
TargetAvgBudget=111.20 | MAE=19.75
[Warmstart 9/15] Step 40/300 | Loss=0.039309 | PredAvgBudget=107.08 |
TargetAvgBudget=113.00 | MAE=18.52
[Warmstart 9/15] Step 60/300 | Loss=0.044799 | PredAvgBudget=111.20 |
TargetAvgBudget=115.33 | MAE=21.67
[Warmstart 9/15] Step 80/300 | Loss=0.048163 | PredAvgBudget=112.20 |
TargetAvgBudget=115.90 | MAE=23.18
[Warmstart 9/15] Step 100/300 | Loss=0.049153 | PredAvgBudget=111.69 |
TargetAvgBudget=116.40 | MAE=23.51
[Warmstart 9/15] Step 120/300 | Loss=0.049302 | PredAvgBudget=112.11 |
TargetAvgBudget=116.07 | MAE=24.04
[Warmstart 9/15] Step 140/300 | Loss=0.052344 | PredAvgBudget=112.16 |
TargetAvgBudget=116.69 | MAE=24.63
[Warmstart 9/15] Step 160/300 | Loss=0.056242 | PredAvgBudget=110.79 |
TargetAvgBudget=116.90 | MAE=24.53
[Warmstart 9/15] Step 180/300 | Loss=0.055241 | PredAvgBudget=110.39 |
TargetAvgBudget=116.40 | MAE=23.95
[Warmstart 9/15] Step 200/300 | Loss=0.054909 | PredAvgBudget=110.27 |
TargetAvgBudget=116.52 | MAE=23.98
[Warmstart 9/15] Step 220/300 | Loss=0.056197 | PredAvgBudget=109.51 |
TargetAvgBudget=116.33 | MAE=23.62
[Warmstart 9/15] Step 240/300 | Loss=0.057148 | PredAvgBudget=110.10 |
TargetAvgBudget=117.20 | MAE=23.99
[Warmstart 9/15] Step 260/300 | Loss=0.055327 | PredAvgBudget=111.66 |
TargetAvgBudget=117.20 | MAE=24.34
[Warmstart 9/15] Step 280/300 | Loss=0.056635 | PredAvgBudget=112.27 |
TargetAvgBudget=118.36 | MAE=24.80
[Warmstart 9/15] Step 300/300 | Loss=0.054970 | PredAvgBudget=113.56 |
TargetAvgBudget=118.44 | MAE=25.36
Warmstart epoch 9 summary | Loss=0.054970 | PredAvgBudget=113.56 |
TargetAvgBudget=118.44

Warm-start deterministic evaluation on test set…
  Deterministic policy eval | Done 10/100 | Acc=0.4000 | AvgBudget=131.00
  Deterministic policy eval | Done 20/100 | Acc=0.3500 | AvgBudget=131.35
  Deterministic policy eval | Done 30/100 | Acc=0.3333 | AvgBudget=130.90
  Deterministic policy eval | Done 40/100 | Acc=0.3000 | AvgBudget=129.80
  Deterministic policy eval | Done 50/100 | Acc=0.3000 | AvgBudget=130.38
```

```
  Deterministic policy eval | Done 60/100 | Acc=0.3000 | AvgBudget=130.38
  Deterministic policy eval | Done 70/100 | Acc=0.3286 | AvgBudget=130.70
  Deterministic policy eval | Done 80/100 | Acc=0.3125 | AvgBudget=130.82
  Deterministic policy eval | Done 90/100 | Acc=0.3111 | AvgBudget=130.64
  Deterministic policy eval | Done 100/100 | Acc=0.3200 | AvgBudget=130.65
Saved new best warm-start policy by reward = 0.3935

[Warmstart 10/15] Step 20/300 | Loss=0.036131 | PredAvgBudget=122.85 |
TargetAvgBudget=116.40 | MAE=27.55
[Warmstart 10/15] Step 40/300 | Loss=0.038588 | PredAvgBudget=120.78 |
TargetAvgBudget=117.00 | MAE=26.43
[Warmstart 10/15] Step 60/300 | Loss=0.046991 | PredAvgBudget=118.07 |
TargetAvgBudget=119.20 | MAE=26.57
[Warmstart 10/15] Step 80/300 | Loss=0.051196 | PredAvgBudget=119.10 |
TargetAvgBudget=120.70 | MAE=28.95
[Warmstart 10/15] Step 100/300 | Loss=0.047700 | PredAvgBudget=120.84 |
TargetAvgBudget=121.04 | MAE=29.08
[Warmstart 10/15] Step 120/300 | Loss=0.046220 | PredAvgBudget=120.87 |
TargetAvgBudget=119.80 | MAE=28.50
[Warmstart 10/15] Step 140/300 | Loss=0.048832 | PredAvgBudget=119.49 |
TargetAvgBudget=120.00 | MAE=28.45
[Warmstart 10/15] Step 160/300 | Loss=0.052598 | PredAvgBudget=117.46 |
TargetAvgBudget=120.42 | MAE=27.92
[Warmstart 10/15] Step 180/300 | Loss=0.053643 | PredAvgBudget=118.35 |
TargetAvgBudget=121.18 | MAE=28.33
[Warmstart 10/15] Step 200/300 | Loss=0.051938 | PredAvgBudget=118.31 |
TargetAvgBudget=120.22 | MAE=28.01
[Warmstart 10/15] Step 220/300 | Loss=0.051167 | PredAvgBudget=116.90 |
TargetAvgBudget=119.36 | MAE=27.03
[Warmstart 10/15] Step 240/300 | Loss=0.051488 | PredAvgBudget=116.27 |
TargetAvgBudget=118.95 | MAE=26.70
[Warmstart 10/15] Step 260/300 | Loss=0.050644 | PredAvgBudget=116.10 |
TargetAvgBudget=118.38 | MAE=26.44
[Warmstart 10/15] Step 280/300 | Loss=0.053581 | PredAvgBudget=115.16 |
TargetAvgBudget=118.61 | MAE=26.35
[Warmstart 10/15] Step 300/300 | Loss=0.053507 | PredAvgBudget=115.06 |
TargetAvgBudget=118.44 | MAE=26.22
Warmstart epoch 10 summary | Loss=0.053507 | PredAvgBudget=115.06 |
TargetAvgBudget=118.44

Warm-start deterministic evaluation on test set…
  Deterministic policy eval | Done 10/100 | Acc=0.3000 | AvgBudget=117.70
  Deterministic policy eval | Done 20/100 | Acc=0.2000 | AvgBudget=118.00
  Deterministic policy eval | Done 30/100 | Acc=0.2000 | AvgBudget=117.30
  Deterministic policy eval | Done 40/100 | Acc=0.2500 | AvgBudget=116.10
  Deterministic policy eval | Done 50/100 | Acc=0.2400 | AvgBudget=116.80
  Deterministic policy eval | Done 60/100 | Acc=0.2500 | AvgBudget=116.68
  Deterministic policy eval | Done 70/100 | Acc=0.2714 | AvgBudget=117.06
```

```
  Deterministic policy eval | Done 80/100 | Acc=0.2625 | AvgBudget=117.11
  Deterministic policy eval | Done 90/100 | Acc=0.2556 | AvgBudget=117.00
  Deterministic policy eval | Done 100/100 | Acc=0.2600 | AvgBudget=116.96
[Warmstart 11/15] Step 20/300 | Loss=0.026573 | PredAvgBudget=100.80 |
TargetAvgBudget=104.00 | MAE=12.00
[Warmstart 11/15] Step 40/300 | Loss=0.051469 | PredAvgBudget=104.28 |
TargetAvgBudget=113.60 | MAE=19.07
[Warmstart 11/15] Step 60/300 | Loss=0.043097 | PredAvgBudget=114.28 |
TargetAvgBudget=116.40 | MAE=22.88
[Warmstart 11/15] Step 80/300 | Loss=0.042728 | PredAvgBudget=115.06 |
TargetAvgBudget=115.40 | MAE=22.96
[Warmstart 11/15] Step 100/300 | Loss=0.058399 | PredAvgBudget=112.20 |
TargetAvgBudget=117.52 | MAE=24.66
[Warmstart 11/15] Step 120/300 | Loss=0.056649 | PredAvgBudget=113.72 |
TargetAvgBudget=118.07 | MAE=24.99
[Warmstart 11/15] Step 140/300 | Loss=0.057637 | PredAvgBudget=115.04 |
TargetAvgBudget=118.09 | MAE=26.06
[Warmstart 11/15] Step 160/300 | Loss=0.059648 | PredAvgBudget=114.01 |
TargetAvgBudget=118.83 | MAE=26.16
[Warmstart 11/15] Step 180/300 | Loss=0.059451 | PredAvgBudget=114.72 |
TargetAvgBudget=119.04 | MAE=26.54
[Warmstart 11/15] Step 200/300 | Loss=0.058792 | PredAvgBudget=114.15 |
TargetAvgBudget=118.70 | MAE=26.15
[Warmstart 11/15] Step 220/300 | Loss=0.057168 | PredAvgBudget=114.52 |
TargetAvgBudget=118.60 | MAE=26.45
[Warmstart 11/15] Step 240/300 | Loss=0.055827 | PredAvgBudget=115.68 |
TargetAvgBudget=118.95 | MAE=26.60
[Warmstart 11/15] Step 260/300 | Loss=0.055022 | PredAvgBudget=115.47 |
TargetAvgBudget=118.32 | MAE=26.26
[Warmstart 11/15] Step 280/300 | Loss=0.053735 | PredAvgBudget=114.76 |
TargetAvgBudget=117.90 | MAE=25.71
[Warmstart 11/15] Step 300/300 | Loss=0.056347 | PredAvgBudget=114.40 |
TargetAvgBudget=118.44 | MAE=25.97
Warmstart epoch 11 summary | Loss=0.056347 | PredAvgBudget=114.40 |
TargetAvgBudget=118.44

Warm-start deterministic evaluation on test set…
  Deterministic policy eval | Done 10/100 | Acc=0.3000 | AvgBudget=108.50
  Deterministic policy eval | Done 20/100 | Acc=0.2500 | AvgBudget=108.85
  Deterministic policy eval | Done 30/100 | Acc=0.2667 | AvgBudget=107.93
  Deterministic policy eval | Done 40/100 | Acc=0.2750 | AvgBudget=106.75
  Deterministic policy eval | Done 50/100 | Acc=0.3000 | AvgBudget=107.50
  Deterministic policy eval | Done 60/100 | Acc=0.3333 | AvgBudget=107.33
  Deterministic policy eval | Done 70/100 | Acc=0.3000 | AvgBudget=107.69
  Deterministic policy eval | Done 80/100 | Acc=0.2750 | AvgBudget=107.75
  Deterministic policy eval | Done 90/100 | Acc=0.2667 | AvgBudget=107.68
  Deterministic policy eval | Done 100/100 | Acc=0.2600 | AvgBudget=107.66
[Warmstart 12/15] Step 20/300 | Loss=0.063711 | PredAvgBudget=111.00 |
```

```
TargetAvgBudget=118.80 | MAE=24.90
[Warmstart 12/15] Step 40/300 | Loss=0.051550 | PredAvgBudget=113.33 |
TargetAvgBudget=117.80 | MAE=24.32
[Warmstart 12/15] Step 60/300 | Loss=0.044453 | PredAvgBudget=115.00 |
TargetAvgBudget=118.13 | MAE=24.33
[Warmstart 12/15] Step 80/300 | Loss=0.047040 | PredAvgBudget=117.59 |
TargetAvgBudget=119.90 | MAE=26.59
[Warmstart 12/15] Step 100/300 | Loss=0.045305 | PredAvgBudget=119.64 |
TargetAvgBudget=119.76 | MAE=27.78
[Warmstart 12/15] Step 120/300 | Loss=0.045872 | PredAvgBudget=118.75 |
TargetAvgBudget=118.93 | MAE=27.30
[Warmstart 12/15] Step 140/300 | Loss=0.042752 | PredAvgBudget=116.99 |
TargetAvgBudget=117.26 | MAE=25.73
[Warmstart 12/15] Step 160/300 | Loss=0.045073 | PredAvgBudget=116.56 |
TargetAvgBudget=117.80 | MAE=25.64
[Warmstart 12/15] Step 180/300 | Loss=0.048983 | PredAvgBudget=116.50 |
TargetAvgBudget=118.24 | MAE=26.03
[Warmstart 12/15] Step 200/300 | Loss=0.049894 | PredAvgBudget=115.46 |
TargetAvgBudget=118.46 | MAE=25.65
[Warmstart 12/15] Step 220/300 | Loss=0.048377 | PredAvgBudget=115.84 |
TargetAvgBudget=117.55 | MAE=25.51
[Warmstart 12/15] Step 240/300 | Loss=0.049107 | PredAvgBudget=115.60 |
TargetAvgBudget=117.92 | MAE=25.56
[Warmstart 12/15] Step 260/300 | Loss=0.049236 | PredAvgBudget=115.13 |
TargetAvgBudget=117.89 | MAE=25.25
[Warmstart 12/15] Step 280/300 | Loss=0.048540 | PredAvgBudget=115.28 |
TargetAvgBudget=117.61 | MAE=25.08
[Warmstart 12/15] Step 300/300 | Loss=0.052244 | PredAvgBudget=114.58 |
TargetAvgBudget=118.44 | MAE=25.57
Warmstart epoch 12 summary | Loss=0.052244 | PredAvgBudget=114.58 |
TargetAvgBudget=118.44

Warm-start deterministic evaluation on test set…
  Deterministic policy eval | Done 10/100 | Acc=0.4000 | AvgBudget=116.70
  Deterministic policy eval | Done 20/100 | Acc=0.3500 | AvgBudget=117.20
  Deterministic policy eval | Done 30/100 | Acc=0.3333 | AvgBudget=116.50
  Deterministic policy eval | Done 40/100 | Acc=0.3250 | AvgBudget=115.12
  Deterministic policy eval | Done 50/100 | Acc=0.3200 | AvgBudget=115.86
  Deterministic policy eval | Done 60/100 | Acc=0.3333 | AvgBudget=115.70
  Deterministic policy eval | Done 70/100 | Acc=0.3714 | AvgBudget=116.11
  Deterministic policy eval | Done 80/100 | Acc=0.3625 | AvgBudget=116.29
  Deterministic policy eval | Done 90/100 | Acc=0.3444 | AvgBudget=116.07
  Deterministic policy eval | Done 100/100 | Acc=0.3300 | AvgBudget=116.00
Saved new best warm-start policy by reward = 0.4067

Saved new best warm-start policy by accuracy = 0.3300

[Warmstart 13/15] Step 20/300 | Loss=0.058748 | PredAvgBudget=126.55 |
```

```
TargetAvgBudget=124.00 | MAE=32.35
[Warmstart 13/15] Step 40/300 | Loss=0.052654 | PredAvgBudget=121.03 |
TargetAvgBudget=120.20 | MAE=28.57
[Warmstart 13/15] Step 60/300 | Loss=0.048253 | PredAvgBudget=118.37 |
TargetAvgBudget=119.47 | MAE=27.47
[Warmstart 13/15] Step 80/300 | Loss=0.049138 | PredAvgBudget=120.22 |
TargetAvgBudget=118.80 | MAE=28.68
[Warmstart 13/15] Step 100/300 | Loss=0.056046 | PredAvgBudget=120.12 |
TargetAvgBudget=121.44 | MAE=29.86
[Warmstart 13/15] Step 120/300 | Loss=0.053781 | PredAvgBudget=121.03 |
TargetAvgBudget=120.93 | MAE=29.74
[Warmstart 13/15] Step 140/300 | Loss=0.052658 | PredAvgBudget=119.06 |
TargetAvgBudget=119.49 | MAE=28.44
[Warmstart 13/15] Step 160/300 | Loss=0.050399 | PredAvgBudget=117.34 |
TargetAvgBudget=118.55 | MAE=26.98
[Warmstart 13/15] Step 180/300 | Loss=0.051487 | PredAvgBudget=116.73 |
TargetAvgBudget=118.13 | MAE=26.73
[Warmstart 13/15] Step 200/300 | Loss=0.049117 | PredAvgBudget=116.81 |
TargetAvgBudget=118.12 | MAE=26.42
[Warmstart 13/15] Step 220/300 | Loss=0.045844 | PredAvgBudget=116.60 |
TargetAvgBudget=117.13 | MAE=25.51
[Warmstart 13/15] Step 240/300 | Loss=0.049547 | PredAvgBudget=116.42 |
TargetAvgBudget=118.70 | MAE=26.15
[Warmstart 13/15] Step 260/300 | Loss=0.049588 | PredAvgBudget=116.73 |
TargetAvgBudget=118.97 | MAE=26.10
[Warmstart 13/15] Step 280/300 | Loss=0.050279 | PredAvgBudget=116.33 |
TargetAvgBudget=119.07 | MAE=26.05
[Warmstart 13/15] Step 300/300 | Loss=0.050024 | PredAvgBudget=115.38 |
TargetAvgBudget=118.44 | MAE=25.50
Warmstart epoch 13 summary | Loss=0.050024 | PredAvgBudget=115.38 |
TargetAvgBudget=118.44

Warm-start deterministic evaluation on test set…
  Deterministic policy eval | Done 10/100 | Acc=0.3000 | AvgBudget=110.30
  Deterministic policy eval | Done 20/100 | Acc=0.3000 | AvgBudget=111.20
  Deterministic policy eval | Done 30/100 | Acc=0.3000 | AvgBudget=110.27
  Deterministic policy eval | Done 40/100 | Acc=0.3000 | AvgBudget=108.65
  Deterministic policy eval | Done 50/100 | Acc=0.3000 | AvgBudget=109.62
  Deterministic policy eval | Done 60/100 | Acc=0.3167 | AvgBudget=109.43
  Deterministic policy eval | Done 70/100 | Acc=0.3000 | AvgBudget=109.84
  Deterministic policy eval | Done 80/100 | Acc=0.2750 | AvgBudget=109.97
  Deterministic policy eval | Done 90/100 | Acc=0.2556 | AvgBudget=109.90
  Deterministic policy eval | Done 100/100 | Acc=0.2500 | AvgBudget=109.88
[Warmstart 14/15] Step 20/300 | Loss=0.037212 | PredAvgBudget=115.70 |
TargetAvgBudget=112.00 | MAE=24.10
[Warmstart 14/15] Step 40/300 | Loss=0.052441 | PredAvgBudget=116.08 |
TargetAvgBudget=118.40 | MAE=26.88
[Warmstart 14/15] Step 60/300 | Loss=0.047166 | PredAvgBudget=118.32 |
```

```
TargetAvgBudget=118.53 | MAE=26.62
[Warmstart 14/15] Step 80/300 | Loss=0.044963 | PredAvgBudget=116.80 |
TargetAvgBudget=117.90 | MAE=25.30
[Warmstart 14/15] Step 100/300 | Loss=0.051394 | PredAvgBudget=116.24 |
TargetAvgBudget=119.36 | MAE=26.22
[Warmstart 14/15] Step 120/300 | Loss=0.049573 | PredAvgBudget=116.49 |
TargetAvgBudget=118.47 | MAE=25.93
[Warmstart 14/15] Step 140/300 | Loss=0.050283 | PredAvgBudget=115.61 |
TargetAvgBudget=119.31 | MAE=26.11
[Warmstart 14/15] Step 160/300 | Loss=0.050387 | PredAvgBudget=116.99 |
TargetAvgBudget=118.97 | MAE=27.20
[Warmstart 14/15] Step 180/300 | Loss=0.047502 | PredAvgBudget=117.41 |
TargetAvgBudget=118.20 | MAE=26.77
[Warmstart 14/15] Step 200/300 | Loss=0.048692 | PredAvgBudget=116.67 |
TargetAvgBudget=118.50 | MAE=26.36
[Warmstart 14/15] Step 220/300 | Loss=0.048707 | PredAvgBudget=115.97 |
TargetAvgBudget=118.09 | MAE=25.86
[Warmstart 14/15] Step 240/300 | Loss=0.050118 | PredAvgBudget=115.48 |
TargetAvgBudget=118.58 | MAE=25.82
[Warmstart 14/15] Step 260/300 | Loss=0.050632 | PredAvgBudget=116.37 |
TargetAvgBudget=118.97 | MAE=26.47
[Warmstart 14/15] Step 280/300 | Loss=0.051599 | PredAvgBudget=115.81 |
TargetAvgBudget=119.10 | MAE=26.27
[Warmstart 14/15] Step 300/300 | Loss=0.049393 | PredAvgBudget=115.07 |
TargetAvgBudget=118.44 | MAE=25.44
Warmstart epoch 14 summary | Loss=0.049393 | PredAvgBudget=115.07 |
TargetAvgBudget=118.44

Warm-start deterministic evaluation on test set…
  Deterministic policy eval | Done 10/100 | Acc=0.4000 | AvgBudget=112.00
  Deterministic policy eval | Done 20/100 | Acc=0.3000 | AvgBudget=112.45
  Deterministic policy eval | Done 30/100 | Acc=0.3000 | AvgBudget=111.33
  Deterministic policy eval | Done 40/100 | Acc=0.3000 | AvgBudget=109.85
  Deterministic policy eval | Done 50/100 | Acc=0.3200 | AvgBudget=110.94
  Deterministic policy eval | Done 60/100 | Acc=0.3333 | AvgBudget=110.68
  Deterministic policy eval | Done 70/100 | Acc=0.3143 | AvgBudget=111.03
  Deterministic policy eval | Done 80/100 | Acc=0.2875 | AvgBudget=111.04
  Deterministic policy eval | Done 90/100 | Acc=0.2778 | AvgBudget=110.92
  Deterministic policy eval | Done 100/100 | Acc=0.2700 | AvgBudget=110.90
[Warmstart 15/15] Step 20/300 | Loss=0.043798 | PredAvgBudget=107.70 |
TargetAvgBudget=112.40 | MAE=18.90
[Warmstart 15/15] Step 40/300 | Loss=0.055470 | PredAvgBudget=111.15 |
TargetAvgBudget=120.60 | MAE=22.35
[Warmstart 15/15] Step 60/300 | Loss=0.049775 | PredAvgBudget=113.35 |
TargetAvgBudget=120.27 | MAE=23.68
[Warmstart 15/15] Step 80/300 | Loss=0.043820 | PredAvgBudget=114.59 |
TargetAvgBudget=118.90 | MAE=23.14
[Warmstart 15/15] Step 100/300 | Loss=0.045294 | PredAvgBudget=113.30 |
```

```
TargetAvgBudget=119.36 | MAE=22.88
[Warmstart 15/15] Step 120/300 | Loss=0.045449 | PredAvgBudget=116.87 |
TargetAvgBudget=119.27 | MAE=25.27
[Warmstart 15/15] Step 140/300 | Loss=0.045944 | PredAvgBudget=118.84 |
TargetAvgBudget=119.89 | MAE=26.44
[Warmstart 15/15] Step 160/300 | Loss=0.046224 | PredAvgBudget=118.09 |
TargetAvgBudget=119.55 | MAE=25.76
[Warmstart 15/15] Step 180/300 | Loss=0.049265 | PredAvgBudget=117.32 |
TargetAvgBudget=120.22 | MAE=25.78
[Warmstart 15/15] Step 200/300 | Loss=0.049722 | PredAvgBudget=116.73 |
TargetAvgBudget=119.42 | MAE=25.59
[Warmstart 15/15] Step 220/300 | Loss=0.052425 | PredAvgBudget=115.69 |
TargetAvgBudget=119.76 | MAE=25.57
[Warmstart 15/15] Step 240/300 | Loss=0.050546 | PredAvgBudget=116.24 |
TargetAvgBudget=119.22 | MAE=25.78
[Warmstart 15/15] Step 260/300 | Loss=0.048273 | PredAvgBudget=116.26 |
TargetAvgBudget=118.57 | MAE=25.25
[Warmstart 15/15] Step 280/300 | Loss=0.051716 | PredAvgBudget=115.37 |
TargetAvgBudget=118.73 | MAE=25.34
[Warmstart 15/15] Step 300/300 | Loss=0.050571 | PredAvgBudget=115.07 |
TargetAvgBudget=118.44 | MAE=25.03
Warmstart epoch 15 summary | Loss=0.050571 | PredAvgBudget=115.07 |
TargetAvgBudget=118.44

Warm-start deterministic evaluation on test set…
  Deterministic policy eval | Done 10/100 | Acc=0.3000 | AvgBudget=114.50
  Deterministic policy eval | Done 20/100 | Acc=0.2500 | AvgBudget=115.30
  Deterministic policy eval | Done 30/100 | Acc=0.2667 | AvgBudget=114.73
  Deterministic policy eval | Done 40/100 | Acc=0.2500 | AvgBudget=113.25
  Deterministic policy eval | Done 50/100 | Acc=0.2600 | AvgBudget=114.00
  Deterministic policy eval | Done 60/100 | Acc=0.2667 | AvgBudget=113.78
  Deterministic policy eval | Done 70/100 | Acc=0.2714 | AvgBudget=114.23
  Deterministic policy eval | Done 80/100 | Acc=0.2625 | AvgBudget=114.50
  Deterministic policy eval | Done 90/100 | Acc=0.2667 | AvgBudget=114.37
  Deterministic policy eval | Done 100/100 | Acc=0.2600 | AvgBudget=114.32
Loading best warm-start policy by accuracy…

Evaluating deterministic policy right after warm-start…
  Deterministic policy eval | Done 10/100 | Acc=0.4000 | AvgBudget=116.70
  Deterministic policy eval | Done 20/100 | Acc=0.3500 | AvgBudget=117.20
  Deterministic policy eval | Done 30/100 | Acc=0.3333 | AvgBudget=116.50
  Deterministic policy eval | Done 40/100 | Acc=0.3250 | AvgBudget=115.12
  Deterministic policy eval | Done 50/100 | Acc=0.3200 | AvgBudget=115.86
  Deterministic policy eval | Done 60/100 | Acc=0.3333 | AvgBudget=115.70
  Deterministic policy eval | Done 70/100 | Acc=0.3714 | AvgBudget=116.11
  Deterministic policy eval | Done 80/100 | Acc=0.3625 | AvgBudget=116.29
  Deterministic policy eval | Done 90/100 | Acc=0.3444 | AvgBudget=116.07
  Deterministic policy eval | Done 100/100 | Acc=0.3300 | AvgBudget=116.00
```

```
{
  "warmstart_deterministic_eval": {
    "accuracy": 0.33,
    "avg_thinking_tokens": 116.0,
    "avg_reward": 0.40669999999999995,
    "avg_budget": 116.0,
    "min_budget": 96.0,
    "max_budget": 130.0,
    "std_budget": 6.797058187186572
  }
}
```


```
Evaluating sampled policy right after warm-start…
  Sampled policy eval | Done 10/100 | Acc=0.3000 | AvgBudget=122.90
  Sampled policy eval | Done 20/100 | Acc=0.2500 | AvgBudget=124.75
  Sampled policy eval | Done 30/100 | Acc=0.3000 | AvgBudget=119.47
  Sampled policy eval | Done 40/100 | Acc=0.2750 | AvgBudget=118.12
  Sampled policy eval | Done 50/100 | Acc=0.3000 | AvgBudget=116.50
  Sampled policy eval | Done 60/100 | Acc=0.3167 | AvgBudget=117.50
  Sampled policy eval | Done 70/100 | Acc=0.3429 | AvgBudget=117.46
  Sampled policy eval | Done 80/100 | Acc=0.3250 | AvgBudget=118.90
  Sampled policy eval | Done 90/100 | Acc=0.3333 | AvgBudget=120.82
  Sampled policy eval | Done 100/100 | Acc=0.3500 | AvgBudget=121.46
```


```
{
  "warmstart_sampled_eval": {
    "accuracy": 0.35,
    "avg_thinking_tokens": 121.46,
    "avg_reward": 0.431427,
    "avg_budget": 121.46,
    "min_budget": 73.0,
    "max_budget": 194.0,
    "std_budget": 29.412385146397085
  }
}
```


```
Starting RL fine-tuning…
[RL 1/5] Step 20/300 | AvgReward=0.5555 | NormRewardMean=0.0000 | Acc=0.4500 |
AvgThinkTokens=139.25 | AvgBudget=139.25 | MinBudget=81.00 | MaxBudget=191.00 |
StdBudget=31.22
Recent budgets: [191, 116, 179, 122, 137, 139, 161, 91, 88, 81]
[RL 1/5] Step 40/300 | AvgReward=0.3994 | NormRewardMean=-0.0000 | Acc=0.3250 |
AvgThinkTokens=136.93 | AvgBudget=136.93 | MinBudget=81.00 | MaxBudget=212.00 |
StdBudget=31.95
Recent budgets: [187, 136, 108, 212, 171, 132, 105, 140, 100, 155]
[RL 1/5] Step 60/300 | AvgReward=0.4101 | NormRewardMean=-0.0000 | Acc=0.3333 |
AvgThinkTokens=132.05 | AvgBudget=132.05 | MinBudget=69.00 | MaxBudget=212.00 |
StdBudget=33.44
Recent budgets: [103, 110, 181, 209, 109, 76, 81, 142, 110, 69]
```

```
[RL 1/5] Step 80/300 | AvgReward=0.3840 | NormRewardMean=-0.0000 | Acc=0.3125 |
AvgThinkTokens=131.66 | AvgBudget=131.66 | MinBudget=69.00 | MaxBudget=212.00 |
StdBudget=35.67
Recent budgets: [212, 90, 118, 155, 144, 100, 152, 119, 94, 189]
[RL 1/5] Step 100/300 | AvgReward=0.4435 | NormRewardMean=0.0000 | Acc=0.3600 |
AvgThinkTokens=130.84 | AvgBudget=130.84 | MinBudget=69.00 | MaxBudget=212.00 |
StdBudget=35.19
Recent budgets: [103, 154, 153, 115, 97, 89, 147, 149, 176, 134]
[RL 1/5] Step 120/300 | AvgReward=0.3997 | NormRewardMean=0.0000 | Acc=0.3250 |
AvgThinkTokens=130.18 | AvgBudget=130.18 | MinBudget=69.00 | MaxBudget=212.00 |
StdBudget=34.24
Recent budgets: [95, 150, 141, 146, 135, 148, 135, 101, 135, 94]
[RL 1/5] Step 140/300 | AvgReward=0.3863 | NormRewardMean=-0.0000 | Acc=0.3143 |
AvgThinkTokens=130.99 | AvgBudget=130.99 | MinBudget=69.00 | MaxBudget=217.00 |
StdBudget=35.36
Recent budgets: [87, 149, 160, 147, 104, 158, 123, 160, 184, 93]
[RL 1/5] Step 160/300 | AvgReward=0.3762 | NormRewardMean=-0.0000 | Acc=0.3063 |
AvgThinkTokens=132.02 | AvgBudget=132.02 | MinBudget=69.00 | MaxBudget=217.00 |
StdBudget=35.44
Recent budgets: [150, 122, 133, 163, 120, 202, 100, 187, 98, 147]
[RL 1/5] Step 180/300 | AvgReward=0.3892 | NormRewardMean=0.0000 | Acc=0.3167 |
AvgThinkTokens=132.40 | AvgBudget=132.40 | MinBudget=69.00 | MaxBudget=218.00 |
StdBudget=35.72
Recent budgets: [151, 103, 142, 197, 108, 156, 133, 139, 173, 119]
[RL 1/5] Step 200/300 | AvgReward=0.3747 | NormRewardMean=0.0000 | Acc=0.3050 |
AvgThinkTokens=131.68 | AvgBudget=131.68 | MinBudget=69.00 | MaxBudget=218.00 |
StdBudget=35.56
Recent budgets: [118, 201, 144, 151, 70, 108, 108, 105, 91, 80]
[RL 1/5] Step 220/300 | AvgReward=0.3797 | NormRewardMean=0.0000 | Acc=0.3091 |
AvgThinkTokens=132.80 | AvgBudget=132.80 | MinBudget=69.00 | MaxBudget=218.00 |
StdBudget=36.24
Recent budgets: [208, 86, 171, 127, 196, 83, 107, 135, 142, 138]
[RL 1/5] Step 240/300 | AvgReward=0.3788 | NormRewardMean=0.0000 | Acc=0.3083 |
AvgThinkTokens=133.25 | AvgBudget=133.25 | MinBudget=69.00 | MaxBudget=218.00 |
StdBudget=36.24
Recent budgets: [141, 114, 155, 128, 95, 125, 97, 190, 111, 192]
[RL 1/5] Step 260/300 | AvgReward=0.3828 | NormRewardMean=0.0000 | Acc=0.3115 |
AvgThinkTokens=133.03 | AvgBudget=133.03 | MinBudget=69.00 | MaxBudget=218.00 |
StdBudget=36.56
Recent budgets: [76, 85, 138, 127, 87, 109, 118, 101, 199, 133]
[RL 1/5] Step 280/300 | AvgReward=0.3907 | NormRewardMean=0.0000 | Acc=0.3179 |
AvgThinkTokens=132.61 | AvgBudget=132.61 | MinBudget=69.00 | MaxBudget=218.00 |
StdBudget=36.90
Recent budgets: [156, 136, 122, 213, 112, 194, 108, 147, 141, 131]
[RL 1/5] Step 300/300 | AvgReward=0.3892 | NormRewardMean=0.0000 | Acc=0.3167 |
AvgThinkTokens=132.64 | AvgBudget=132.64 | MinBudget=69.00 | MaxBudget=218.00 |
StdBudget=36.59
Recent budgets: [111, 81, 120, 114, 140, 130, 127, 133, 131, 159]
```

```
================================================================================
==========
RL Epoch 1 summary
Train Accuracy        : 0.3167
Train Avg ThinkTokens : 132.64
Train Avg Reward      : 0.3892
Train Avg Budget      : 132.64
Train Min Budget      : 69.00
Train Max Budget      : 218.00
Train Std Budget      : 36.59

Deterministic policy evaluation on test set…
  Deterministic policy eval | Done 10/100 | Acc=0.4000 | AvgBudget=116.70
  Deterministic policy eval | Done 20/100 | Acc=0.3500 | AvgBudget=117.20
  Deterministic policy eval | Done 30/100 | Acc=0.3333 | AvgBudget=116.50
  Deterministic policy eval | Done 40/100 | Acc=0.3250 | AvgBudget=115.12
  Deterministic policy eval | Done 50/100 | Acc=0.3200 | AvgBudget=115.86
  Deterministic policy eval | Done 60/100 | Acc=0.3333 | AvgBudget=115.70
  Deterministic policy eval | Done 70/100 | Acc=0.3714 | AvgBudget=116.11
  Deterministic policy eval | Done 80/100 | Acc=0.3625 | AvgBudget=116.29
  Deterministic policy eval | Done 90/100 | Acc=0.3444 | AvgBudget=116.07
  Deterministic policy eval | Done 100/100 | Acc=0.3300 | AvgBudget=116.00
Test Accuracy         : 0.3300
Test Avg ThinkTokens  : 116.00
Test Avg Reward       : 0.4067
Test Avg Budget       : 116.00
Test Min Budget       : 96.00
Test Max Budget       : 130.00
Test Std Budget       : 6.80
================================================================================
==========

Saved new best RL models by reward = 0.4067

Saved new best RL policy by accuracy = 0.3300

[RL 2/5] Step 20/300 | AvgReward=0.4310 | NormRewardMean=-0.0000 | Acc=0.3500 |
AvgThinkTokens=130.75 | AvgBudget=130.75 | MinBudget=71.00 | MaxBudget=177.00 |
StdBudget=26.53
Recent budgets: [161, 138, 143, 94, 146, 133, 149, 95, 137, 150]
[RL 2/5] Step 40/300 | AvgReward=0.5247 | NormRewardMean=0.0000 | Acc=0.4250 |
AvgThinkTokens=131.80 | AvgBudget=131.80 | MinBudget=70.00 | MaxBudget=212.00 |
StdBudget=31.05
Recent budgets: [113, 100, 157, 163, 141, 158, 105, 97, 110, 91]
[RL 2/5] Step 60/300 | AvgReward=0.4520 | NormRewardMean=-0.0000 | Acc=0.3667 |
AvgThinkTokens=127.62 | AvgBudget=127.62 | MinBudget=70.00 | MaxBudget=212.00 |
StdBudget=31.83
```

```
Recent budgets: [150, 82, 137, 88, 97, 193, 94, 119, 76, 154]
[RL 2/5] Step 80/300 | AvgReward=0.4309 | NormRewardMean=-0.0000 | Acc=0.3500 |
AvgThinkTokens=131.65 | AvgBudget=131.65 | MinBudget=70.00 | MaxBudget=217.00 |
StdBudget=32.03
Recent budgets: [144, 157, 126, 217, 168, 96, 95, 111, 125, 178]
[RL 2/5] Step 100/300 | AvgReward=0.4309 | NormRewardMean=-0.0000 | Acc=0.3500 |
AvgThinkTokens=131.95 | AvgBudget=131.95 | MinBudget=70.00 | MaxBudget=217.00 |
StdBudget=31.66
Recent budgets: [133, 125, 88, 158, 102, 153, 141, 125, 197, 106]
[RL 2/5] Step 120/300 | AvgReward=0.3892 | NormRewardMean=-0.0000 | Acc=0.3167 |
AvgThinkTokens=133.43 | AvgBudget=133.43 | MinBudget=70.00 | MaxBudget=217.00 |
StdBudget=32.11
Recent budgets: [139, 147, 103, 193, 123, 201, 195, 157, 108, 164]
[RL 2/5] Step 140/300 | AvgReward=0.3862 | NormRewardMean=0.0000 | Acc=0.3143 |
AvgThinkTokens=133.87 | AvgBudget=133.87 | MinBudget=70.00 | MaxBudget=217.00 |
StdBudget=33.63
Recent budgets: [154, 177, 136, 188, 188, 93, 182, 217, 148, 107]
[RL 2/5] Step 160/300 | AvgReward=0.3606 | NormRewardMean=0.0000 | Acc=0.2938 |
AvgThinkTokens=132.16 | AvgBudget=132.16 | MinBudget=70.00 | MaxBudget=217.00 |
StdBudget=33.84
Recent budgets: [147, 145, 181, 86, 109, 116, 168, 71, 111, 90]
[RL 2/5] Step 180/300 | AvgReward=0.3823 | NormRewardMean=0.0000 | Acc=0.3111 |
AvgThinkTokens=130.90 | AvgBudget=130.90 | MinBudget=70.00 | MaxBudget=217.00 |
StdBudget=34.07
Recent budgets: [115, 84, 108, 143, 85, 86, 176, 102, 178, 94]
[RL 2/5] Step 200/300 | AvgReward=0.3810 | NormRewardMean=-0.0000 | Acc=0.3100 |
AvgThinkTokens=130.55 | AvgBudget=130.55 | MinBudget=70.00 | MaxBudget=217.00 |
StdBudget=34.44
Recent budgets: [80, 117, 139, 180, 90, 144, 171, 168, 98, 100]
[RL 2/5] Step 220/300 | AvgReward=0.3741 | NormRewardMean=-0.0000 | Acc=0.3045 |
AvgThinkTokens=131.11 | AvgBudget=131.11 | MinBudget=70.00 | MaxBudget=217.00 |
StdBudget=34.29
Recent budgets: [93, 125, 125, 149, 141, 188, 104, 178, 175, 110]
[RL 2/5] Step 240/300 | AvgReward=0.3893 | NormRewardMean=0.0000 | Acc=0.3167 |
AvgThinkTokens=131.50 | AvgBudget=131.50 | MinBudget=70.00 | MaxBudget=217.00 |
StdBudget=35.59
Recent budgets: [108, 100, 110, 86, 103, 75, 217, 89, 155, 122]
[RL 2/5] Step 260/300 | AvgReward=0.3829 | NormRewardMean=0.0000 | Acc=0.3115 |
AvgThinkTokens=131.33 | AvgBudget=131.33 | MinBudget=70.00 | MaxBudget=217.00 |
StdBudget=35.20
Recent budgets: [107, 72, 145, 139, 111, 190, 84, 97, 183, 124]
[RL 2/5] Step 280/300 | AvgReward=0.3997 | NormRewardMean=0.0000 | Acc=0.3250 |
AvgThinkTokens=131.78 | AvgBudget=131.78 | MinBudget=70.00 | MaxBudget=219.00 |
StdBudget=35.81
Recent budgets: [108, 206, 194, 125, 85, 91, 182, 158, 81, 165]
[RL 2/5] Step 300/300 | AvgReward=0.3892 | NormRewardMean=-0.0000 | Acc=0.3167 |
AvgThinkTokens=132.00 | AvgBudget=132.00 | MinBudget=70.00 | MaxBudget=219.00 |
StdBudget=35.81
```

```
Recent budgets: [147, 155, 99, 73, 121, 123, 114, 100, 154, 111]

================================================================================
==========
RL Epoch 2 summary
Train Accuracy        : 0.3167
Train Avg ThinkTokens : 132.00
Train Avg Reward      : 0.3892
Train Avg Budget      : 132.00
Train Min Budget      : 70.00
Train Max Budget      : 219.00
Train Std Budget      : 35.81

Deterministic policy evaluation on test set…
  Deterministic policy eval | Done 10/100 | Acc=0.4000 | AvgBudget=116.70
  Deterministic policy eval | Done 20/100 | Acc=0.3500 | AvgBudget=117.20
  Deterministic policy eval | Done 30/100 | Acc=0.3333 | AvgBudget=116.50
  Deterministic policy eval | Done 40/100 | Acc=0.3250 | AvgBudget=115.12
  Deterministic policy eval | Done 50/100 | Acc=0.3200 | AvgBudget=115.86
  Deterministic policy eval | Done 60/100 | Acc=0.3333 | AvgBudget=115.70
  Deterministic policy eval | Done 70/100 | Acc=0.3714 | AvgBudget=116.11
  Deterministic policy eval | Done 80/100 | Acc=0.3625 | AvgBudget=116.29
  Deterministic policy eval | Done 90/100 | Acc=0.3444 | AvgBudget=116.07
  Deterministic policy eval | Done 100/100 | Acc=0.3300 | AvgBudget=116.00
Test Accuracy         : 0.3300
Test Avg ThinkTokens  : 116.00
Test Avg Reward       : 0.4067
Test Avg Budget       : 116.00
Test Min Budget       : 96.00
Test Max Budget       : 130.00
Test Std Budget       : 6.80
================================================================================
==========

[RL 3/5] Step 20/300 | AvgReward=0.2444 | NormRewardMean=-0.0000 | Acc=0.2000 |
AvgThinkTokens=111.25 | AvgBudget=111.25 | MinBudget=78.00 | MaxBudget=177.00 |
StdBudget=23.39
Recent budgets: [119, 113, 78, 92, 114, 105, 121, 86, 126, 157]
[RL 3/5] Step 40/300 | AvgReward=0.2444 | NormRewardMean=-0.0000 | Acc=0.2000 |
AvgThinkTokens=112.42 | AvgBudget=112.42 | MinBudget=73.00 | MaxBudget=177.00 |
StdBudget=27.07
Recent budgets: [124, 90, 99, 141, 83, 103, 75, 98, 98, 83]
[RL 3/5] Step 60/300 | AvgReward=0.2651 | NormRewardMean=-0.0000 | Acc=0.2167 |
AvgThinkTokens=113.72 | AvgBudget=113.72 | MinBudget=73.00 | MaxBudget=177.00 |
StdBudget=26.29
Recent budgets: [129, 136, 112, 93, 95, 103, 99, 124, 107, 109]
[RL 3/5] Step 80/300 | AvgReward=0.2754 | NormRewardMean=0.0000 | Acc=0.2250 |
AvgThinkTokens=116.91 | AvgBudget=116.91 | MinBudget=73.00 | MaxBudget=190.00 |
```

```
StdBudget=26.98
Recent budgets: [105, 154, 118, 158, 113, 145, 124, 131, 102, 190]
[RL 3/5] Step 100/300 | AvgReward=0.3192 | NormRewardMean=0.0000 | Acc=0.2600 |
AvgThinkTokens=116.35 | AvgBudget=116.35 | MinBudget=72.00 | MaxBudget=190.00 |
StdBudget=28.05
Recent budgets: [166, 81, 118, 93, 165, 94, 85, 97, 187, 77]
[RL 3/5] Step 120/300 | AvgReward=0.2961 | NormRewardMean=-0.0000 | Acc=0.2417 |
AvgThinkTokens=119.39 | AvgBudget=119.39 | MinBudget=72.00 | MaxBudget=202.00 |
StdBudget=29.39
Recent budgets: [97, 105, 152, 119, 144, 169, 94, 152, 117, 165]
[RL 3/5] Step 140/300 | AvgReward=0.2887 | NormRewardMean=-0.0000 | Acc=0.2357 |
AvgThinkTokens=119.06 | AvgBudget=119.06 | MinBudget=72.00 | MaxBudget=202.00 |
StdBudget=29.56
Recent budgets: [156, 123, 78, 114, 125, 181, 140, 123, 97, 162]
[RL 3/5] Step 160/300 | AvgReward=0.2987 | NormRewardMean=-0.0000 | Acc=0.2437 |
AvgThinkTokens=119.12 | AvgBudget=119.12 | MinBudget=72.00 | MaxBudget=202.00 |
StdBudget=28.95
Recent budgets: [103, 102, 132, 104, 161, 95, 96, 116, 125, 103]
[RL 3/5] Step 180/300 | AvgReward=0.3065 | NormRewardMean=-0.0000 | Acc=0.2500 |
AvgThinkTokens=119.62 | AvgBudget=119.62 | MinBudget=72.00 | MaxBudget=202.00 |
StdBudget=29.08
Recent budgets: [174, 126, 102, 95, 178, 111, 177, 107, 108, 83]
[RL 3/5] Step 200/300 | AvgReward=0.3190 | NormRewardMean=-0.0000 | Acc=0.2600 |
AvgThinkTokens=120.27 | AvgBudget=120.27 | MinBudget=72.00 | MaxBudget=202.00 |
StdBudget=29.17
Recent budgets: [77, 97, 117, 113, 86, 92, 151, 164, 94, 107]
[RL 3/5] Step 220/300 | AvgReward=0.3235 | NormRewardMean=0.0000 | Acc=0.2636 |
AvgThinkTokens=120.00 | AvgBudget=120.00 | MinBudget=69.00 | MaxBudget=202.00 |
StdBudget=30.19
Recent budgets: [91, 99, 99, 127, 89, 88, 175, 127, 83, 108]
[RL 3/5] Step 240/300 | AvgReward=0.3325 | NormRewardMean=-0.0000 | Acc=0.2708 |
AvgThinkTokens=120.53 | AvgBudget=120.53 | MinBudget=69.00 | MaxBudget=202.00 |
StdBudget=30.19
Recent budgets: [177, 143, 90, 160, 137, 124, 81, 150, 188, 100]
[RL 3/5] Step 260/300 | AvgReward=0.3305 | NormRewardMean=-0.0000 | Acc=0.2692 |
AvgThinkTokens=120.42 | AvgBudget=120.42 | MinBudget=69.00 | MaxBudget=202.00 |
StdBudget=30.06
Recent budgets: [137, 75, 105, 174, 81, 128, 124, 96, 87, 85]
[RL 3/5] Step 280/300 | AvgReward=0.3377 | NormRewardMean=-0.0000 | Acc=0.2750 |
AvgThinkTokens=120.84 | AvgBudget=120.84 | MinBudget=69.00 | MaxBudget=202.00 |
StdBudget=30.08
Recent budgets: [81, 108, 104, 135, 168, 144, 94, 77, 125, 119]
[RL 3/5] Step 300/300 | AvgReward=0.3398 | NormRewardMean=0.0000 | Acc=0.2767 |
AvgThinkTokens=121.46 | AvgBudget=121.46 | MinBudget=69.00 | MaxBudget=202.00 |
StdBudget=30.03
Recent budgets: [165, 175, 135, 79, 145, 149, 122, 124, 121, 133]

================================================================================
```

```
==========
RL Epoch 3 summary
Train Accuracy         : 0.2767
Train Avg ThinkTokens  : 121.46
Train Avg Reward       : 0.3398
Train Avg Budget       : 121.46
Train Min Budget       : 69.00
Train Max Budget       : 202.00
Train Std Budget       : 30.03

Deterministic policy evaluation on test set…
  Deterministic policy eval | Done 10/100 | Acc=0.4000 | AvgBudget=116.70
  Deterministic policy eval | Done 20/100 | Acc=0.3500 | AvgBudget=117.20
  Deterministic policy eval | Done 30/100 | Acc=0.3333 | AvgBudget=116.50
  Deterministic policy eval | Done 40/100 | Acc=0.3250 | AvgBudget=115.12
  Deterministic policy eval | Done 50/100 | Acc=0.3200 | AvgBudget=115.86
  Deterministic policy eval | Done 60/100 | Acc=0.3333 | AvgBudget=115.70
  Deterministic policy eval | Done 70/100 | Acc=0.3714 | AvgBudget=116.11
  Deterministic policy eval | Done 80/100 | Acc=0.3625 | AvgBudget=116.29
  Deterministic policy eval | Done 90/100 | Acc=0.3444 | AvgBudget=116.07
  Deterministic policy eval | Done 100/100 | Acc=0.3300 | AvgBudget=116.00
Test Accuracy          : 0.3300
Test Avg ThinkTokens   : 116.00
Test Avg Reward        : 0.4067
Test Avg Budget        : 116.00
Test Min Budget        : 96.00
Test Max Budget        : 130.00
Test Std Budget        : 6.80
================================================================================
==========

[RL 4/5] Step 20/300 | AvgReward=0.4310 | NormRewardMean=-0.0000 | Acc=0.3500 |
AvgThinkTokens=129.70 | AvgBudget=129.70 | MinBudget=75.00 | MaxBudget=189.00 |
StdBudget=32.24
Recent budgets: [163, 122, 99, 75, 114, 94, 103, 171, 119, 150]
[RL 4/5] Step 40/300 | AvgReward=0.4001 | NormRewardMean=-0.0000 | Acc=0.3250 |
AvgThinkTokens=122.50 | AvgBudget=122.50 | MinBudget=75.00 | MaxBudget=202.00 |
StdBudget=32.41
Recent budgets: [90, 112, 91, 104, 84, 164, 138, 107, 100, 121]
[RL 4/5] Step 60/300 | AvgReward=0.4107 | NormRewardMean=-0.0000 | Acc=0.3333 |
AvgThinkTokens=120.23 | AvgBudget=120.23 | MinBudget=75.00 | MaxBudget=202.00 |
StdBudget=30.23
Recent budgets: [123, 123, 188, 137, 75, 89, 114, 110, 80, 104]
[RL 4/5] Step 80/300 | AvgReward=0.4315 | NormRewardMean=0.0000 | Acc=0.3500 |
AvgThinkTokens=120.71 | AvgBudget=120.71 | MinBudget=75.00 | MaxBudget=202.00 |
StdBudget=29.87
Recent budgets: [112, 126, 124, 76, 81, 121, 84, 165, 133, 119]
[RL 4/5] Step 100/300 | AvgReward=0.4315 | NormRewardMean=0.0000 | Acc=0.3500 |
```

```
AvgThinkTokens=120.68 | AvgBudget=120.68 | MinBudget=75.00 | MaxBudget=202.00 |
StdBudget=29.19
Recent budgets: [116, 115, 132, 88, 113, 101, 129, 158, 130, 103]
[RL 4/5] Step 120/300 | AvgReward=0.4105 | NormRewardMean=0.0000 | Acc=0.3333 |
AvgThinkTokens=122.73 | AvgBudget=122.73 | MinBudget=75.00 | MaxBudget=202.00 |
StdBudget=29.15
Recent budgets: [103, 135, 82, 175, 155, 115, 177, 140, 155, 141]
[RL 4/5] Step 140/300 | AvgReward=0.4403 | NormRewardMean=0.0000 | Acc=0.3571 |
AvgThinkTokens=122.47 | AvgBudget=122.47 | MinBudget=75.00 | MaxBudget=202.00 |
StdBudget=28.14
Recent budgets: [122, 145, 98, 140, 96, 120, 89, 102, 163, 139]
[RL 4/5] Step 160/300 | AvgReward=0.4079 | NormRewardMean=-0.0000 | Acc=0.3312 |
AvgThinkTokens=123.33 | AvgBudget=123.33 | MinBudget=75.00 | MaxBudget=202.00 |
StdBudget=28.83
Recent budgets: [86, 110, 147, 111, 130, 134, 146, 85, 174, 127]
[RL 4/5] Step 180/300 | AvgReward=0.3966 | NormRewardMean=-0.0000 | Acc=0.3222 |
AvgThinkTokens=122.60 | AvgBudget=122.60 | MinBudget=72.00 | MaxBudget=202.00 |
StdBudget=28.74
Recent budgets: [85, 72, 104, 170, 122, 136, 171, 135, 147, 112]
[RL 4/5] Step 200/300 | AvgReward=0.4001 | NormRewardMean=0.0000 | Acc=0.3250 |
AvgThinkTokens=122.43 | AvgBudget=122.43 | MinBudget=72.00 | MaxBudget=202.00 |
StdBudget=28.32
Recent budgets: [151, 85, 135, 107, 138, 164, 119, 91, 132, 125]
[RL 4/5] Step 220/300 | AvgReward=0.3973 | NormRewardMean=0.0000 | Acc=0.3227 |
AvgThinkTokens=122.00 | AvgBudget=122.00 | MinBudget=72.00 | MaxBudget=202.00 |
StdBudget=28.37
Recent budgets: [155, 105, 115, 105, 142, 122, 100, 143, 173, 121]
[RL 4/5] Step 240/300 | AvgReward=0.3897 | NormRewardMean=0.0000 | Acc=0.3167 |
AvgThinkTokens=121.82 | AvgBudget=121.82 | MinBudget=72.00 | MaxBudget=202.00 |
StdBudget=28.35
Recent budgets: [146, 114, 105, 85, 85, 124, 88, 105, 114, 96]
[RL 4/5] Step 260/300 | AvgReward=0.3881 | NormRewardMean=0.0000 | Acc=0.3154 |
AvgThinkTokens=122.49 | AvgBudget=122.49 | MinBudget=72.00 | MaxBudget=202.00 |
StdBudget=28.60
Recent budgets: [121, 181, 123, 96, 111, 82, 152, 104, 166, 163]
[RL 4/5] Step 280/300 | AvgReward=0.3778 | NormRewardMean=-0.0000 | Acc=0.3071 |
AvgThinkTokens=122.26 | AvgBudget=122.26 | MinBudget=72.00 | MaxBudget=202.00 |
StdBudget=28.17
Recent budgets: [133, 104, 132, 123, 133, 119, 130, 115, 104, 109]
[RL 4/5] Step 300/300 | AvgReward=0.3689 | NormRewardMean=-0.0000 | Acc=0.3000 |
AvgThinkTokens=122.86 | AvgBudget=122.86 | MinBudget=72.00 | MaxBudget=202.00 |
StdBudget=28.26
Recent budgets: [106, 110, 105, 84, 177, 142, 170, 149, 108, 117]

================================================================================
==========
RL Epoch 4 summary
Train Accuracy        : 0.3000
```

```
Train Avg ThinkTokens : 122.86
Train Avg Reward      : 0.3689
Train Avg Budget      : 122.86
Train Min Budget      : 72.00
Train Max Budget      : 202.00
Train Std Budget      : 28.26

Deterministic policy evaluation on test set…
  Deterministic policy eval | Done 10/100 | Acc=0.4000 | AvgBudget=116.70
  Deterministic policy eval | Done 20/100 | Acc=0.3500 | AvgBudget=117.20
  Deterministic policy eval | Done 30/100 | Acc=0.3333 | AvgBudget=116.50
  Deterministic policy eval | Done 40/100 | Acc=0.3250 | AvgBudget=115.12
  Deterministic policy eval | Done 50/100 | Acc=0.3200 | AvgBudget=115.86
  Deterministic policy eval | Done 60/100 | Acc=0.3333 | AvgBudget=115.70
  Deterministic policy eval | Done 70/100 | Acc=0.3714 | AvgBudget=116.11
  Deterministic policy eval | Done 80/100 | Acc=0.3625 | AvgBudget=116.29
  Deterministic policy eval | Done 90/100 | Acc=0.3444 | AvgBudget=116.07
  Deterministic policy eval | Done 100/100 | Acc=0.3300 | AvgBudget=116.00
Test Accuracy         : 0.3300
Test Avg ThinkTokens  : 116.00
Test Avg Reward       : 0.4067
Test Avg Budget       : 116.00
Test Min Budget       : 96.00
Test Max Budget       : 130.00
Test Std Budget       : 6.80
================================================================================
==========

[RL 5/5] Step 20/300 | AvgReward=0.2440 | NormRewardMean=-0.0000 | Acc=0.2000 |
AvgThinkTokens=120.35 | AvgBudget=120.35 | MinBudget=74.00 | MaxBudget=177.00 |
StdBudget=30.24
Recent budgets: [74, 119, 116, 171, 137, 113, 155, 139, 114, 79]
[RL 5/5] Step 40/300 | AvgReward=0.1503 | NormRewardMean=0.0000 | Acc=0.1250 |
AvgThinkTokens=118.17 | AvgBudget=118.17 | MinBudget=74.00 | MaxBudget=177.00 |
StdBudget=29.85
Recent budgets: [153, 145, 125, 105, 106, 82, 96, 106, 79, 79]
[RL 5/5] Step 60/300 | AvgReward=0.1609 | NormRewardMean=-0.0000 | Acc=0.1333 |
AvgThinkTokens=114.92 | AvgBudget=114.92 | MinBudget=74.00 | MaxBudget=177.00 |
StdBudget=27.64
Recent budgets: [90, 103, 120, 157, 122, 87, 85, 93, 134, 136]
[RL 5/5] Step 80/300 | AvgReward=0.1818 | NormRewardMean=0.0000 | Acc=0.1500 |
AvgThinkTokens=114.88 | AvgBudget=114.88 | MinBudget=72.00 | MaxBudget=194.00 |
StdBudget=29.16
Recent budgets: [93, 104, 174, 132, 148, 93, 117, 98, 92, 129]
[RL 5/5] Step 100/300 | AvgReward=0.2317 | NormRewardMean=0.0000 | Acc=0.1900 |
AvgThinkTokens=116.79 | AvgBudget=116.79 | MinBudget=72.00 | MaxBudget=194.00 |
StdBudget=28.71
Recent budgets: [147, 110, 112, 104, 152, 100, 152, 124, 102, 166]
```

```
[RL 5/5] Step 120/300 | AvgReward=0.2337 | NormRewardMean=-0.0000 | Acc=0.1917 |
AvgThinkTokens=116.83 | AvgBudget=116.83 | MinBudget=72.00 | MaxBudget=194.00 |
StdBudget=27.24
Recent budgets: [103, 127, 80, 119, 112, 101, 107, 131, 101, 111]
[RL 5/5] Step 140/300 | AvgReward=0.2531 | NormRewardMean=-0.0000 | Acc=0.2071 |
AvgThinkTokens=117.40 | AvgBudget=117.40 | MinBudget=72.00 | MaxBudget=194.00 |
StdBudget=26.85
Recent budgets: [90, 99, 152, 94, 138, 136, 139, 94, 104, 85]
[RL 5/5] Step 160/300 | AvgReward=0.2753 | NormRewardMean=0.0000 | Acc=0.2250 |
AvgThinkTokens=118.31 | AvgBudget=118.31 | MinBudget=72.00 | MaxBudget=194.00 |
StdBudget=27.82
Recent budgets: [107, 151, 73, 126, 81, 101, 161, 185, 102, 172]
[RL 5/5] Step 180/300 | AvgReward=0.2718 | NormRewardMean=0.0000 | Acc=0.2222 |
AvgThinkTokens=118.91 | AvgBudget=118.91 | MinBudget=72.00 | MaxBudget=194.00 |
StdBudget=27.79
Recent budgets: [101, 98, 93, 157, 82, 91, 118, 153, 79, 133]
[RL 5/5] Step 200/300 | AvgReward=0.2878 | NormRewardMean=-0.0000 | Acc=0.2350 |
AvgThinkTokens=119.04 | AvgBudget=119.04 | MinBudget=72.00 | MaxBudget=194.00 |
StdBudget=27.96
Recent budgets: [114, 93, 101, 98, 146, 75, 157, 154, 98, 103]
[RL 5/5] Step 220/300 | AvgReward=0.2895 | NormRewardMean=0.0000 | Acc=0.2364 |
AvgThinkTokens=119.41 | AvgBudget=119.41 | MinBudget=72.00 | MaxBudget=194.00 |
StdBudget=27.76
Recent budgets: [169, 138, 127, 124, 122, 120, 98, 104, 165, 95]
[RL 5/5] Step 240/300 | AvgReward=0.3013 | NormRewardMean=-0.0000 | Acc=0.2458 |
AvgThinkTokens=119.26 | AvgBudget=119.26 | MinBudget=72.00 | MaxBudget=194.00 |
StdBudget=27.95
Recent budgets: [140, 118, 113, 137, 190, 83, 85, 110, 72, 168]
[RL 5/5] Step 260/300 | AvgReward=0.2921 | NormRewardMean=-0.0000 | Acc=0.2385 |
AvgThinkTokens=119.06 | AvgBudget=119.06 | MinBudget=72.00 | MaxBudget=194.00 |
StdBudget=27.41
Recent budgets: [109, 108, 95, 136, 109, 82, 142, 130, 129, 124]
[RL 5/5] Step 280/300 | AvgReward=0.2842 | NormRewardMean=0.0000 | Acc=0.2321 |
AvgThinkTokens=118.93 | AvgBudget=118.93 | MinBudget=72.00 | MaxBudget=194.00 |
StdBudget=27.14
Recent budgets: [130, 110, 160, 162, 147, 88, 108, 97, 105, 98]
[RL 5/5] Step 300/300 | AvgReward=0.2982 | NormRewardMean=-0.0000 | Acc=0.2433 |
AvgThinkTokens=118.80 | AvgBudget=118.80 | MinBudget=68.00 | MaxBudget=194.00 |
StdBudget=27.25
Recent budgets: [79, 174, 68, 118, 101, 114, 111, 114, 115, 112]

================================================================================
==========
RL Epoch 5 summary
Train Accuracy        : 0.2433
Train Avg ThinkTokens : 118.80
Train Avg Reward      : 0.2982
Train Avg Budget      : 118.80
```

```
Train Min Budget      : 68.00
Train Max Budget      : 194.00
Train Std Budget      : 27.25

Deterministic policy evaluation on test set…
  Deterministic policy eval | Done 10/100 | Acc=0.4000 | AvgBudget=116.70
  Deterministic policy eval | Done 20/100 | Acc=0.3500 | AvgBudget=117.20
  Deterministic policy eval | Done 30/100 | Acc=0.3333 | AvgBudget=116.50
  Deterministic policy eval | Done 40/100 | Acc=0.3250 | AvgBudget=115.12
  Deterministic policy eval | Done 50/100 | Acc=0.3200 | AvgBudget=115.86
  Deterministic policy eval | Done 60/100 | Acc=0.3333 | AvgBudget=115.70
  Deterministic policy eval | Done 70/100 | Acc=0.3714 | AvgBudget=116.11
  Deterministic policy eval | Done 80/100 | Acc=0.3625 | AvgBudget=116.29
  Deterministic policy eval | Done 90/100 | Acc=0.3444 | AvgBudget=116.07
  Deterministic policy eval | Done 100/100 | Acc=0.3300 | AvgBudget=116.00
Test Accuracy         : 0.3300
Test Avg ThinkTokens  : 116.00
Test Avg Reward       : 0.4067
Test Avg Budget       : 116.00
Test Min Budget       : 96.00
Test Max Budget       : 130.00
Test Std Budget       : 6.80
================================================================================
==========

Loading best saved policy/value for final evaluation…
Loading best RL policy by accuracy…

Final deterministic policy evaluation…
  Deterministic policy eval | Done 10/100 | Acc=0.4000 | AvgBudget=116.70
  Deterministic policy eval | Done 20/100 | Acc=0.3500 | AvgBudget=117.20
  Deterministic policy eval | Done 30/100 | Acc=0.3333 | AvgBudget=116.50
  Deterministic policy eval | Done 40/100 | Acc=0.3250 | AvgBudget=115.12
  Deterministic policy eval | Done 50/100 | Acc=0.3200 | AvgBudget=115.86
  Deterministic policy eval | Done 60/100 | Acc=0.3333 | AvgBudget=115.70
  Deterministic policy eval | Done 70/100 | Acc=0.3714 | AvgBudget=116.11
  Deterministic policy eval | Done 80/100 | Acc=0.3625 | AvgBudget=116.29
  Deterministic policy eval | Done 90/100 | Acc=0.3444 | AvgBudget=116.07
  Deterministic policy eval | Done 100/100 | Acc=0.3300 | AvgBudget=116.00
{
  "final_deterministic_eval": {
    "accuracy": 0.33,
    "avg_thinking_tokens": 116.0,
    "avg_reward": 0.40669999999999995,
    "avg_budget": 116.0,
    "min_budget": 96.0,
    "max_budget": 130.0,
    "std_budget": 6.797058187186572
```

```
  }
}

Final sampled policy evaluation…
  Sampled policy eval | Done 10/100 | Acc=0.3000 | AvgBudget=113.30
  Sampled policy eval | Done 20/100 | Acc=0.2500 | AvgBudget=123.40
  Sampled policy eval | Done 30/100 | Acc=0.2667 | AvgBudget=120.00
  Sampled policy eval | Done 40/100 | Acc=0.3000 | AvgBudget=122.12
  Sampled policy eval | Done 50/100 | Acc=0.3200 | AvgBudget=120.08
  Sampled policy eval | Done 60/100 | Acc=0.3333 | AvgBudget=119.93
  Sampled policy eval | Done 70/100 | Acc=0.3571 | AvgBudget=121.11
  Sampled policy eval | Done 80/100 | Acc=0.3250 | AvgBudget=119.67
  Sampled policy eval | Done 90/100 | Acc=0.3333 | AvgBudget=119.39
  Sampled policy eval | Done 100/100 | Acc=0.3200 | AvgBudget=119.54
{
  "final_sampled_eval": {
    "accuracy": 0.32,
    "avg_thinking_tokens": 119.54,
    "avg_reward": 0.39402299999999996,
    "avg_budget": 119.54,
    "min_budget": 70.0,
    "max_budget": 182.0,
    "std_budget": 26.535794693206384
  }
}

Saved final policy to two_stage_continuous_budget_policy_v9_final.pt
Saved final value net to two_stage_value_net_v9_final.pt
```

[ ]:
```
AI Disclouser: I have taken the help of ChatGPT while writing the code
```